

Deterministic DNNF Is Not Polynomially Closed Under Negation

Guy Van den Broeck*
Computer Science Department
University of California, Los Angeles

Abstract

We prove¹ that the logic circuit language d-DNNF is not polynomially closed under negation, resolving a question first posed by Darwiche and Marquis in 2001 [23]. d-DNNF circuits restrict negation to inputs, AND gates to disjoint variable sets, and OR gates to mutually exclusive cases, allowing efficient counting of satisfying assignments. The seminal knowledge compilation map’s transformation table left only this closure problem open [24, Table 7]. More precisely, we prove that there are Boolean functions on m variables with d-DNNFs of size $O(m)$ whose negations require quasipolynomially larger general DNNFs of size $m^{\Omega(\log m)}$, even when determinism is dropped from the output representation. We give a detailed tutorial proof, adapting the unambiguous DNF construction and matrix argument of Pabbaraju [58], and an end-to-end Lean formalization of the nonclosure theorem. As a logic circuit corollary, extensions of d-DNNF and its structured variants with internal negation, proposed repeatedly in the literature [11, 39, 54, 57, 78], admit superpolynomially smaller representations than DNNF. As an arithmetic circuit corollary, representing the missing multilinear monomials of a polynomial can require a quasipolynomial increase in monotone circuit size. Further corollaries show that converting decomposable probabilistic circuits to nonnegative form, complementing unambiguous grammars, and replacing unambiguous finite automata by deterministic pushdown automata can each require a quasipolynomial size increase.

*This paper builds on contributions by Oliver Broadrick. He has not yet agreed to be an author, but has agreed to the circulation of this note.

¹This proof was developed with substantial AI assistance. At the time of writing, scientific standards for AI-assisted theoretical results are not yet settled. We have chosen to take three steps:

1. We have carefully checked the claims and proofs in this paper and take full responsibility for their correctness.
2. We have presented the proof as a detailed tutorial requiring little background beyond undergraduate mathematics. Footnotes throughout the proof highlight connections to more advanced topics. The tutorial proof aims to make the reasoning and the ideas behind the construction accessible to human readers, motivated by Tao’s view that “*A proof that no human can properly explain should be viewed as incomplete, even if it has been formally verified*” [74, Section 8].
3. Readers can check the nonclosure claim by inspecting a short formal trust boundary: the Lean definitions and theorem statement explained in Appendix A. If these express the intended claim, the machine-checked proof establishes it, assuming the correctness of Lean’s kernel. Readers who also want to formally check that the theorem holds for the reasons given in the tutorial proof can compare the two proofs: each statement links to its Lean formalization.

Contents

1	Introduction	3
2	Constructing the Boolean function	5
2.1	Mutually exclusive DNF terms	5
2.2	A short unambiguous DNF	8
2.3	Replacing each Boolean coordinate by four bits	10
2.4	Linear encoding and the small d-DNNF	11
3	Proving that zero squares are small	12
3.1	A matrix supported on one-inputs	12
3.2	Parity vectors and their local inner products	14
3.3	Computing the eigenvalues of K	15
3.4	Bounding the least eigenvalue	17
3.5	Zero squares are small	19
4	Proving the DNNF lower bound	21
4.1	Choosing the encoder for the lower bound	21
4.2	From zero squares to a rectangle lower bound	23
5	Corollaries	27
5.1	Logic circuits	27
5.2	Arithmetic circuits	28
5.3	Probabilistic circuits	29
5.4	Automata and grammars	29
6	Conclusion	31
	References	32
A	The Lean trust boundary	36
A.1	Nodes and circuits	36
A.2	The computed function and the circuit classes	38
A.3	The exact Lean claim	39
B	Proof details	40
B.1	Padding the hard function	40
B.2	An algebraic derivation of the local inner products	41
B.3	From a DNNF to a balanced rectangle cover	41
C	Negation of decision-DNNFs	43

1 Introduction

Knowledge compilation studies representations of Boolean functions that support efficient queries and transformations [24, 67]. A basic transformation is negation: how much larger must a representation become when a function is replaced by its complement? Many circuit representations, including FBDDs and OBDDs [80], SDDs [22], and TDDs [14], support negation in polynomial time. We study the more general circuit language d-DNNF.

A negation normal form (NNF) circuit is a directed acyclic graph whose inputs are Boolean literals and constants and whose internal gates are AND or OR gates with any number of inputs. The circuit is *decomposable* when the inputs to every AND gate mention disjoint sets of variables. It is *deterministic* when distinct inputs to every OR gate cannot be true on the same assignment. DNNF imposes the first condition, while d-DNNF imposes both. These restrictions make counting satisfying assignments polynomial in the circuit size [24]. Over the past decades, d-DNNF has become the language for exact model counting. Many solvers construct these circuits explicitly [21, 42, 47, 51, 59] or implicitly through search [37]. Weighted model counting on these circuits also underlies exact inference in probabilistic graphical models [25, 26, 66] and probabilistic programs [30, 36].

Darwiche and Marquis asked whether d-DNNF is polynomially closed under negation: can every d-DNNF be replaced by one for its complement with only a polynomial increase in size [23]? In their seminal knowledge compilation map’s transformation table [24, Table 7], the corresponding entries for (smooth) d-DNNF were the only open question marks. The question has been repeatedly restated in subsequent work [1, 2, 4, 13, 15, 27, 29, 37, 39, 40, 43, 46, 48, 52, 53, 59, 60, 70, 71, 73, 76–78].

Vinall-Smeeth [76] made important progress toward resolving this question by proving nonclosure under negation for structured d-DNNFs, a case also left open in earlier work [8, 60]. In these circuits, a variable tree constrains how AND gates partition the variables. Vinall-Smeeth’s lower bound applies to structured DNNFs for the negation with any variable tree. This did not settle the original question, since the negations might still have admitted small unstructured d-DNNFs.

Numerous papers have also proposed allowing internal negation in d-DNNF, directly or through signed operations: cd-PDAG [78], d-DNNF[−] [39], d-D [54], POGs [11], and st-DASCs [57]. With negation closure still unresolved, it remained unclear whether such extensions admit superpolynomially smaller representations than classic d-DNNF [3, 28, 54, 78].

Our result. We resolve the d-DNNF negation question by constructing functions with small d-DNNFs whose negations require quasipolynomially larger DNNFs. The lower bound allows the output circuit to abandon both determinism and any common variable tree. To state the result precisely, we define the size $|C|$ of a circuit C to be its number of edges plus one. Thus a single literal or constant has size one. For a Boolean function f , DNNF-size(f) and d-DNNF-size(f) denote the least size of a DNNF, respectively a d-DNNF, computing f .

Non-closure under negation

✓ Separation: `nonclosure_under_negation`

Theorem 1. For arbitrarily large m , there is a Boolean function $H : \{0, 1\}^m \rightarrow \{0, 1\}$ such that

$$\text{d-DNNF-size}(H) = O(m), \quad \text{DNNF-size}(\neg H) = m^{\Omega(\log m)}.$$

Thus the negations require at least quasipolynomial size even as general DNNFs. In particular, d-DNNF is not polynomially closed under negation.

Smoothness requires the inputs of each OR gate to mention the same variables and can be imposed with polynomial overhead [24, 68]. Thus smooth d-DNNF is also not polynomially closed under negation, settling its open question mark in the knowledge compilation map.

Consequences. The separation explains what is gained by allowing negation inside a circuit. Adding just one NOT gate to our small d-DNNF gives a small representation of its complement. Eliminating that gate requires a superpolynomial increase in size, even if the resulting circuit need only be a DNNF (Corollary 33). Thus such extensions admit superpolynomially smaller representations than DNNF.

Arithmetic circuits compute polynomials using addition and multiplication gates [69]. Valiant showed that subtraction can yield exponential savings over monotone arithmetic circuits [75], and later work obtains such separations even with syntactically multilinear circuits [17, 19]. Our arithmetic corollary concerns a specific subtraction: complementing a monomial set. We give a multilinear polynomial with 0/1 coefficients and a small *monotone* circuit, using only nonnegative constants. One subtraction computes its missing multilinear monomials, but every monotone circuit for them is quasipolynomially larger, regardless of their positive coefficients (Corollary 35).

Probabilistic circuits represent probability distributions using sum and product gates [20]. The analogous question is whether negative weights give more succinct representations of these distributions. Here *decomposability* imposes the same disjoint-variable condition at products. Loconte et al. [49] prove exponential savings over structured-decomposable nonnegative circuits. We put a uniform distribution on the complement of the models of our small d-DNNF. One subtraction suffices for a small structured-decomposable circuit, but every nonnegative decomposable circuit for this distribution is quasipolynomially larger, even when its decompositions need not follow a fixed variable tree (Corollary 36).

Reading assignments as words connects our result to automata and grammars [38]. Our functions yield finite languages with small unambiguous finite automata, but their complements require quasipolynomially larger context-free grammars. Even for the original languages, replacing unambiguous choice by a deterministic pushdown automaton requires a quasipolynomial increase in size: access to a stack does not make up for the restriction to deterministic computation (Corollaries 37 and 38).

The upper bound also holds for *unambiguous OBDDs*, which read variables in a fixed order and allow nondeterministic choices with at most one accepting computation per assignment [7]. In particular, there are unambiguous OBDDs whose negations have no DNNF representations of polynomially related size (Corollary 34). The small OBDDs exist in every variable order.

Proof idea. Our proof starts from progress on a related question: how much can negation increase unambiguous DNF width? A DNF is *unambiguous* if no assignment satisfies two of its terms, and its *width* is the largest number of literals in a term. Pabbaraju [58] constructs unambiguous DNFs of width $O(n)$ whose negations require DNF width $\Omega(n^2)$, achieving the optimal quadratic gap without the logarithmic losses in earlier bounds [5]. We adapt his construction and matrix argument. The challenge is to turn a lower bound against short DNF terms into a lower bound against DNNFs, which can combine and share many smaller computations.

We do this by constructing functions L_n with the following bounds.

The hard function before padding

✓ Separation: `negation_separation`

Lemma 2. *There is a family of Boolean functions L_n on $\Theta(n^2)$ variables such that*

$$\text{d-DNNF-size}(L_n) = 2^{O(n)}, \quad \text{DNNF-size}(\neg L_n) = 2^{\Omega(n^2)}.$$

Padding gives Theorem 1: it makes the d-DNNF size at most the new input length and preserves the lower bound. A circuit for the padded function's negation would give one for $\neg L_n$ by fixing the unused variables. Appendix B.1 gives the details.

Construction. Our function consists of three layers, shown in Figure 1. We start with an unambiguous DNF f_n adapted from Pabbaraju's construction. Each of its input variables is replaced by a copy of a four-bit Boolean function Q , giving an intermediate function F_n . Finally, take a Boolean input vector x . A binary matrix E produces Ex , with arithmetic modulo 2; the resulting bits supply the four-bit inputs to the copies of Q . We define $L_n(x) = F_n(Ex)$.

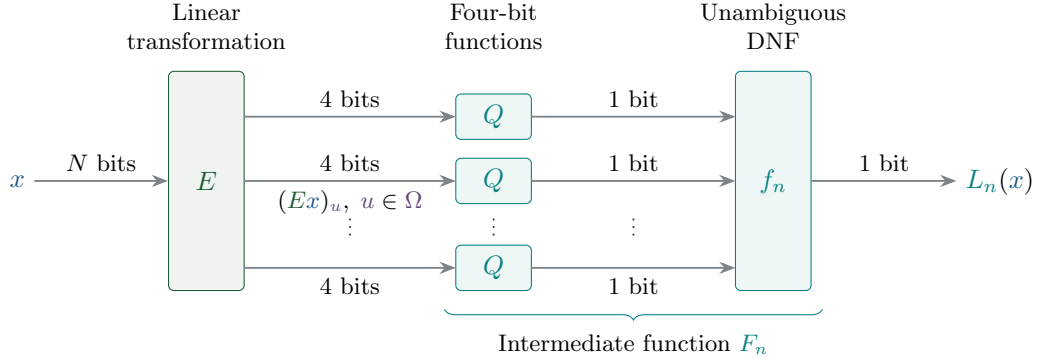


Figure 1: The three layers of L_n , evaluated from left to right. Here $N = \Theta(n^2)$ is the number of input variables. The linear transformation E supplies one four-bit block to each copy of Q ; the DNF f_n combines their one-bit outputs. The copies of Q together with f_n form the intermediate function F_n . Section 2.1 defines the block index $u \in \Omega$.

The DNF layer supplies the small representation. It has $2^{O(n)}$ mutually exclusive terms, each containing $O(n)$ literals. Even after replacing its variables by four-bit functions fed by the linear transformation, each term has a d-DNNF of size $2^{O(n)}$. The terms remain mutually exclusive, so their disjunction is a d-DNNF of size $2^{O(n)}$. Section 2 constructs the three layers and proves this upper bound.

Lower bound. A small DNNF for $\neg L_n$ would cover the zeros of L_n with few *balanced rectangles* [10]. A rectangle independently combines assignments on two groups partitioning the variables; it is balanced when each group contains at least a third of the variables. We exhibit a fixed set of zeros such that every balanced rectangle containing only zeros of L_n contains only an exponentially small fraction of this set. Covering this set therefore requires many rectangles, even when different rectangles use different partitions.

The four-bit layer makes it possible to bound the zeros accessible to a rectangle. Following Pabbaraju, we construct a nonnegative matrix K indexed by inputs to F_n such that

$$F_n(a + b) = 0 \implies K(a, b) = 0,$$

where addition is modulo 2 in each coordinate. The freedom to assign weights to the four-bit inputs allows us to control the eigenvalues of this matrix. We use that control to show that a set I with $F_n(a + b) = 0$ for all $a, b \in I$ must occupy an exponentially small fraction of the input space. We call $I \times I$ a *zero square*; it is also all zero in K . Section 3 proves this bound.

To apply this bound across different variable partitions, we choose E so that varying any set of at least a third of the input variables, while fixing the others, reaches a constant fraction of all possible joint inputs to the Q blocks. Section 4.1 proves that such a transformation exists.

Section 4 combines the matrix bound with the encoder property to establish the required bound on the fraction of zeros covered by each rectangle.

Sections 2, 3, and 4 present the proof as a detailed tutorial, with each statement linked to its Lean formalization. Section 5 develops the corollaries in more detail, and Section 6 concludes.

2 Constructing the Boolean function

Fix an integer $n \geq 1$. We construct L_n and prove the small d-DNNF upper bound in Lemma 2.

2.1 Mutually exclusive DNF terms

The terms below adapt a construction of Pabbaraju [58].

Definition 3. Let $[n]_0 = \{0, \dots, n-1\}$. The set of Boolean coordinates is the Cartesian product, or n -by- n grid,

$$\Omega = \underbrace{[n]_0}_{\text{bucket index}} \times \underbrace{[n]_0}_{\text{label}}, \quad |\Omega| = n^2.$$

We partition this grid into its n rows, which we call *buckets*. For each $i \in [n]_0$, let

$$B_i = \{i\} \times [n]_0, \quad \Omega = \bigcup_{i \in [n]_0} B_i.$$

Thus a Boolean coordinate is a pair (i, a) : the first component names its bucket, and the second is a label within that bucket. Every bucket has its own copy of the same label set $[n]_0$. We choose n ways of ranking those common labels:

$$(\pi_r)_{r \in [n]_0}, \quad \pi_r : [n]_0 \longrightarrow [n]_0.$$

Think of each permutation π_r as a label order that can be applied to the labels in any bucket. The number $\pi_r(a)$ is the rank of label a in that order.

For every bucket index i and every nonempty set of labels $S \subseteq [n]_0$, define one DNF term. Its positive positions are the copies of those labels in B_i :

$$P_{i,S} = \{(i, a) : a \in S\} \subseteq B_i.$$

Thus S itself contains labels $a \in [n]_0$, not coordinate pairs. The map $S \mapsto P_{i,S}$ selects exactly the coordinates (i, a) of the already defined bucket B_i whose label a belongs to S .

For every label order π and every nonempty label set S , define the cutoff

$$M_\pi(S) = \min_{a \in S} \pi(a).$$

Now fix $j \neq i$. For the DNF term indexed by (i, S) , we call B_i the *source* bucket and B_j a *target* bucket. Some coordinates of B_j will index negative literals in the DNF term defined later; we denote their set by $A_{i \rightarrow j}(S)$.

The target's label order π_j determines the cutoff $M_{\pi_j}(S)$. The source's label order π_i orders the labels in the target bucket B_j . The set $A_{i \rightarrow j}(S)$ is the prefix ending at that cutoff:

$$A_{i \rightarrow j}(S) = \{(j, b) \in B_j : \pi_i(b) \leq M_{\pi_j}(S)\}.$$

Example Computing one target-bucket prefix

Suppose S contains the labels $\{0, 2, 5\}$ and the target label order gives

$$\pi_j(0) = 4, \quad \pi_j(2) = 1, \quad \pi_j(5) = 3.$$

Then $M_{\pi_j}(S) = 1$. If the labels $\{3, 1\}$ have ranks 0 and 1 under π_i , then

$$A_{i \rightarrow j}(S) = \{(j, 3), (j, 1)\}.$$

The other values of the label orders do not affect this set.

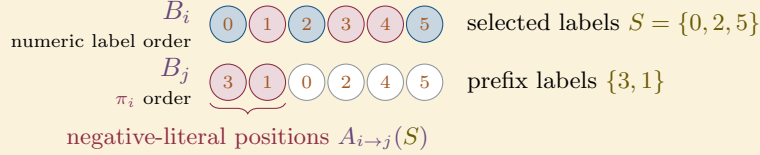
The full negative set is

$$N_{i,S} = (B_i \setminus P_{i,S}) \dot{\cup} \bigcup_{j \neq i} A_{i \rightarrow j}(S).$$

It is disjoint from $P_{i,S}$: the first part is its complement inside B_i , and all the sets $A_{i \rightarrow j}(S)$ lie in other buckets. The corresponding term of the Boolean DNF is the conjunction

$$T_{i,S}(y) = \bigwedge_{u \in P_{i,S}} y_u \wedge \bigwedge_{v \in N_{i,S}} \neg y_v. \quad (2.1)$$

Example One DNF term: source bucket and one target bucket



Circle numbers are labels. Blue requires the variable to be 1, red requires it to be 0, and white leaves it free. Since B_j is shown in π_i order, its red segment is exactly $A_{i \rightarrow j}(S)$.

The negative literals enforce two restrictions. Inside B_i , they make the DNF term select exactly the pattern $P_{i,S}$. The prefixes in the other buckets prevent DNF terms from different buckets from being true together. We first make the latter statement precise.

Distinct outer terms are mutually exclusive

✓ OuterDNF: distinct_terms_conflict

Lemma 4. For every choice of the label orders, any two distinct term indices (i, S) and (i', S') , where $\emptyset \neq S, S' \subseteq [n]_0$, satisfy

$$(i, S) \neq (i', S') \implies (P_{i,S} \cap N_{i',S'} \neq \emptyset) \text{ or } (P_{i',S'} \cap N_{i,S} \neq \emptyset). \quad (2.2)$$

Example Two terms that clash

Take $n = 6$ and distinct bucket indices i, i' , and list the labels in increasing rank under each label order:

$$\pi_i : 3 \prec 1 \prec 0 \prec 2 \prec 4 \prec 5, \quad \pi_{i'} : 4 \prec 2 \prec 1 \prec 5 \prec 0 \prec 3.$$

Compare $T_{i,S}$ for $S = \{0, 2, 5\}$ with $T_{i',S'}$ for $S' = \{0, 2, 4\}$. The label in S of smallest $\pi_{i'}$ -rank is $a_* = 2$, and the label in S' of smallest π_i -rank is $b_* = 0$. Thus $\pi_{i'}(a_*) = 1 \leq 2 = \pi_i(b_*)$. The corresponding intersection is

$$A_{i' \rightarrow i}(S') \cap P_{i,S} = \{(i, 4), (i, 2), (i, 1)\} \cap \{(i, 0), (i, 2), (i, 5)\} = \{(i, 2)\}.$$

Thus the two DNF terms contain the opposite literals $y_{(i,2)}$ and $\neg y_{(i,2)}$.

Proof of Lemma 4. First consider two terms in the same bucket. Their label sets S and S' are distinct, so one of them contains a label omitted by the other; say $a \in S \setminus S'$. The coordinate (i, a) is positive in $T_{i,S}$, and it is negative in $T_{i,S'}$ because every unselected label of the own bucket is negative there.

Now consider $T_{i,S}$ and $T_{i',S'}$ with $i \neq i'$. Choose $a_* \in S$ of smallest $\pi_{i'}$ -rank and $b_* \in S'$ of smallest π_i -rank. Because $a_* \in S$ and $b_* \in S'$, the coordinate (i, a_*) is positive in the first term, and (i', b_*) is positive in the second. We show that at least one of these coordinates is negative in the other term.

The chosen labels attain the minima in the negative-prefix definitions:

$$\begin{aligned} A_{i \rightarrow i'}(S) &= \{(i', b) : b \in [n]_0, \pi_i(b) \leq M_{\pi_{i'}}(S) = \pi_{i'}(a_*)\}, \\ A_{i' \rightarrow i}(S') &= \{(i, a) : a \in [n]_0, \pi_{i'}(a) \leq M_{\pi_i}(S') = \pi_i(b_*)\}. \end{aligned}$$

Substituting $b = b_*$ in the first set and $a = a_*$ in the second gives the two possible conflicts:

$$\pi_i(b_*) \leq \pi_{i'}(a_*) \implies (i', b_*) \in A_{i \rightarrow i'}(S) \cap P_{i',S'},$$

$$\pi_{i'}(a_*) \leq \pi_i(b_*) \implies (i, a_*) \in A_{i' \rightarrow i}(S') \cap P_{i,S}.$$

At least one of these inequalities holds, so the two terms require opposite values at a common coordinate. This proves Lemma 4. $\square$

2.2 A short unambiguous DNF

Our final DNF retains the terms indexed by label sets with at least $n/3$ labels.² In a random label order, a label from such a large set usually appears near the beginning, so its cutoff is usually small. Across n independent label orders, a large total cutoff length is sufficiently unlikely that some choice of label orders keeps every retained term short.

One choice of label orders makes every term short

✓ OuterDNF: every_term_short

Lemma 5. *There is a choice of the n label orders such that, for every $i \in [n]_0$ and every $S \subseteq [n]_0$ with $3 \cdot |S| \geq n$,*

$$|P_{i,S}| + |N_{i,S}| \leq 10n. \quad (2.3)$$

Proof. For any fixed choice of label orders, the source bucket contributes exactly n literals: $|S|$ positive and $n - |S|$ negative. For each target bucket $j \neq i$, the prefix definition gives

$$|A_{i \rightarrow j}(S)| = \min_{a \in S} \pi_j(a) + 1 = M_{\pi_j}(S) + 1.$$

To bound the widths for every source bucket at once, sum all n cutoff lengths, including the one with $j = i$:

$$C(S) = \sum_{j \in [n]_0} (M_{\pi_j}(S) + 1).$$

The width of each term is therefore bounded by

$$|P_{i,S}| + |N_{i,S}| = n + \sum_{j \neq i} |A_{i \rightarrow j}(S)| \leq n + C(S). \quad (2.4)$$

It is enough to choose label orders with $C(S) \leq 9n$ for every label set with at least $n/3$ labels. We will find such orders by sampling them at random.

Fix a label set $S \subseteq [n]_0$ with $3 \cdot |S| \geq n$, and now sample π uniformly from all label orders. Then the already defined $M_\pi(S)$ is a random variable, and the event $M_\pi(S) \geq t$ says that none of the first t positions, namely ranks $0, 1, \dots, t-1$, belongs to S . Write $s = |S|$ and first consider an integer $0 \leq t \leq n - s$. A label order is determined by listing the labels at ranks $0, 1, \dots, n-1$ in turn, and every list of n distinct labels occurs once. The lists whose first t entries avoid S are counted by choosing those entries first: there are $n - s$ choices for rank 0, then $n - s - 1$ for rank 1, and so on down to $n - s - t + 1$ for rank $t-1$, and the remaining $n - t$ ranks can be filled in $(n - t)!$ ways. Dividing by the total $n! = n(n-1) \cdots (n-t+1)(n-t)!$ gives the exact value³

$$\Pr_\pi[M_\pi(S) \geq t] = \prod_{j=0}^{t-1} \frac{n-s-j}{n-j}.$$

For each j in the product,

$$\frac{n-s-j}{n-j} = 1 - \frac{s}{n-j} \leq 1 - \frac{s}{n} = \frac{n-s}{n}.$$

Since we have assumed that $s \geq n/3$, each factor is at most $2/3$. For $t > n - s$, the probability is zero: there are not enough labels outside S to fill the first t positions. Thus for every integer $t \geq 0$,

$$\Pr_\pi[M_\pi(S) \geq t] \leq \left(\frac{n-s}{n}\right)^t \leq \left(\frac{2}{3}\right)^t. \quad (2.5)$$

²Pabbaraju's construction instead uses label sets of exactly $\lfloor n/2 \rfloor$ labels [58].

³ $M_\pi(S)$ is the minimum order statistic of a uniformly random s -element subset of the n ranks, and its tail is the survival probability for a negative hypergeometric random variable.

To turn (2.5) into an exponential-moment bound, put $X = M_\pi(S)$. Recall the finite geometric-sum identity

$$1 + r + \dots + r^X = \sum_{t=0}^X r^t = \frac{r^{X+1} - 1}{r - 1}.$$

First isolate the power that we need:

$$r^{X+1} = 1 + (r - 1) \sum_{t=0}^X r^t.$$

Now set $r = 5/4$:

$$\left(\frac{5}{4}\right)^{X+1} = 1 + \frac{1}{4} \sum_{t=0}^X \left(\frac{5}{4}\right)^t = \frac{5}{4} + \frac{1}{4} \sum_{t=1}^X \left(\frac{5}{4}\right)^t.$$

Taking expectations, the t th summand appears exactly on the event $X \geq t$. By (2.5), that event has probability at most $(2/3)^t$. Hence the resulting series is geometric:

$$\begin{aligned} \mathbb{E}_\pi \left[\left(\frac{5}{4}\right)^{X+1} \right] &= \frac{5}{4} + \frac{1}{4} \sum_{t \geq 1} \left(\frac{5}{4}\right)^t \Pr_\pi[X \geq t] \\ &\leq \frac{5}{4} + \frac{1}{4} \sum_{t \geq 1} \left(\frac{5}{4} \cdot \frac{2}{3}\right)^t = \frac{5}{4} + \frac{1}{4} \sum_{t \geq 1} \left(\frac{5}{6}\right)^t \\ &= \frac{5}{4} + \frac{1}{4} \cdot \frac{5/6}{1 - 5/6} = \frac{5}{2} \leq 3. \end{aligned} \tag{2.6}$$

Choose the n label orders $(\pi_j)_{j \in [n]_0}$ independently. Write $\pi = (\pi_0, \dots, \pi_{n-1})$ for this random tuple. For each fixed S , the quantity $C(S)$ is now a random variable: it is the total of the n cutoff lengths determined by the sampled label orders. Exponentiating the defining sum turns it into a product of independent factors. Substituting $X = M_{\pi_j}(S)$ in (2.6) bounds the expectation of each factor by 3, so

$$\mathbb{E}_\pi \left[\left(\frac{5}{4}\right)^{C(S)} \right] = \prod_{j \in [n]_0} \mathbb{E}_{\pi_j} \left[\left(\frac{5}{4}\right)^{M_{\pi_j}(S)+1} \right] \leq 3^n.$$

Markov's inequality says that $\Pr[Y > a] \leq \mathbb{E}[Y]/a$ for every nonnegative random variable Y and every $a > 0$. Apply it here with $Y = (5/4)^{C(S)}$ and $a = (5/4)^{9n}$:

$$\Pr_\pi[C(S) > 9n] \leq \frac{\mathbb{E}_\pi[(5/4)^{C(S)}]}{(5/4)^{9n}} \leq \frac{3^n}{(5/4)^{9n}}.$$

The subscript records that only the tuple of label orders is random. For each fixed S with $3 \cdot |S| \geq n$, the inequality $C(S) > 9n$ defines one bad event for the same random tuple of label orders. There are fewer than 2^n such events, so a union bound gives

$$\Pr_\pi[\exists S \subseteq [n]_0 : 3 \cdot |S| \geq n \text{ and } C(S) > 9n] \leq \frac{2^n 3^n}{(5/4)^{9n}} = \left(\frac{6}{(5/4)^9}\right)^n < 1.$$

The final inequality holds because $(5/4)^9 > 6$. Therefore some single tuple of label orders avoids every bad event: for that tuple, $C(S) \leq 9n$ for every label set with at least $n/3$ labels. Fix this tuple. Equation (2.4) now bounds every retained term's width by $n + 9n = 10n$. This proves Lemma 5. $\square$

Definition 6. Fix a tuple of label orders satisfying Lemma 5. Define the Boolean function

$$f_n(y) = \bigvee_{i \in [n]_0} \bigvee_{\substack{S \subseteq [n]_0 \\ 3 \cdot |S| \geq n}} T_{i,S}(y), \quad y \in \{0, 1\}^\Omega. \quad (2.7)$$

The DNF has at most $n2^n = 2^{O(n)}$ terms, each of width $O(n)$. Lemma 4 says that no input satisfies two different terms, so this DNF is unambiguous. Also every label set in the family is nonempty, so every term contains a positive literal. Hence

$$f_n(0^\Omega) = 0. \quad (2.8)$$

Each term has a particularly simple satisfying input. If $\mathbf{1}_{P_{i,S}}$ denotes the Boolean vector that is 1 exactly on $P_{i,S}$, then it satisfies $T_{i,S}$: it is 1 on every positive position and 0 everywhere else. Therefore

$$f_n(\mathbf{1}_{P_{i,S}}) = 1. \quad (2.9)$$

2.3 Replacing each Boolean coordinate by four bits

We construct F_n from f_n by replacing each input bit with a function Q of four bits. The four-bit inputs belong to

$$V = \mathbb{F}_2^2 \times \mathbb{F}_2^2.$$

Here $\mathbb{F}_2 = \{0, 1\}$ is the two-element field, so a value in V is a four-bit string viewed as a pair of two-bit halves. Addition is bitwise XOR; every value is therefore its own negative. Write one four-bit string as $(w, w') \in V$, where $w, w' \in \mathbb{F}_2^2$, and define the inner product of its two halves:⁴

$$Q((w, w')) = w_1 w'_1 + w_2 w'_2 \bmod 2. \quad (2.10)$$

The values of Q

The cell in row w and column w' shows $Q((w, w'))$. The row and column together specify one four-bit input. The framed corner is the input $(00, 00)$.

$w \backslash w'$	00	01	10	11
00	0	0	0	0
01	0	1	0	1
10	0	0	1	1
11	0	1	1	0

Now apply Q across all n^2 coordinates. Write

$$G = V^\Omega.$$

An element of G assigns one four-bit string to every coordinate, and vectors in G are added coordinatewise. Thus G is a vector space over $\mathbb{F}_2$ of dimension $4n^2$, and every vector is its own negative.

For a single full vector $\mathbf{z} = (z_u)_{u \in \Omega} \in G$, with four-bit entries $z_u \in V$, define $F_n : G \rightarrow \{0, 1\}$ by applying Q separately to each coordinate and feeding the resulting Boolean vector to f_n :

$$F_n(\mathbf{z}) = f_n\left(\left(Q(z_u)\right)_{u \in \Omega}\right). \quad (2.11)$$

Since $Q((00, 00)) = 0$, Equations (2.8) and (2.11) give

$$F_n(\mathbf{0}) = f_n(0^\Omega) = 0. \quad (2.12)$$

⁴Inner-product gadgets also appear in communication-complexity lifting theorems [18]. We use a fixed four-bit gadget and a direct spectral argument.

2.4 Linear encoding and the small d-DNNF

An input to F_n is a vector in G containing $4n^2$ bits. We use a binary matrix to turn a longer Boolean assignment into such a vector. The extra input bits will let us choose the matrix so that varying any third of them, with the others fixed, still reaches a constant fraction of all possible inputs to F_n (Section 4.1). We first prove the upper bound for every choice of this matrix.

The linear encoder and the Boolean function

Lean: Encoder

Definition 7. Set $N = 60n^2$. Let E be any $4n^2$ -by- N matrix over $\mathbb{F}_2$. For $x \in \{0, 1\}^N$, we view Ex as an element of G by grouping its entries into four-bit blocks indexed by Ω . Define

$$L_n(x) = F_n(Ex).$$

Figure 1 shows how the encoder E , the copies of Q , and the outer DNF f_n compose to compute L_n . By (2.11), a literal of an outer term inspects the bit $Q((Ex)_u)$ for one coordinate $u \in \Omega$. After substitution, different literals may depend on the same input bits x_j , so conjoining circuits for the individual literals need not be decomposable. We instead evaluate the entire substituted term with one ordered binary decision diagram (OBDD): a branching program that reads the input bits in one fixed order.

One OBDD for a substituted term

✓ PositiveSide: term_obdd

Lemma 8. Fix a term of the outer DNF for f_n that inspects k distinct coordinates. After substituting $Q((Ex)_u)$ for each outer coordinate u , the term is computed by an OBDD with at most $(N + 1)16^k$ nodes.

Proof. Read the input bits in the order $x_1, \dots, x_N$. Consider the level after the first t bits have been read. Write (p, y) for an input with prefix p and remaining bits y . Filling the other positions with zeros, linearity gives

$$E(p, y) = E(p, 0) + E(0, y).$$

Thus the bit inspected by an outer literal at coordinate u is

$$Q((E(p, 0))_u + (E(0, y))_u).$$

This bit may depend on every input bit. But two prefixes with the same four-bit contributions $(E(p, 0))_u$ at all k inspected coordinates give the same result for every remaining assignment y . They therefore leave the same function of the unread bits and can share an OBDD node, as in Figure 2. There are at most 16^k possible tuples of these contributions, hence at most 16^k distinct remaining functions at this level.

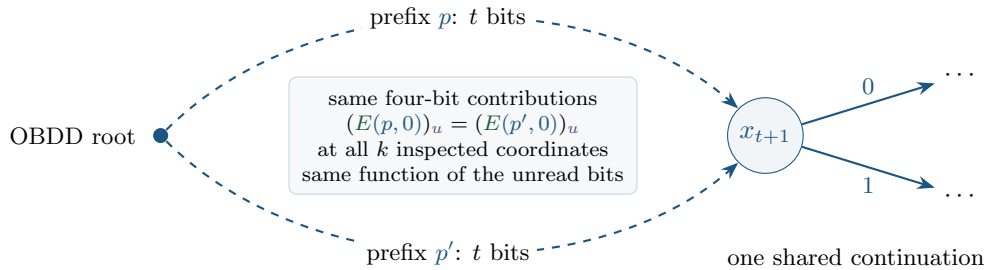


Figure 2: Why different prefixes can share an OBDD node. The dashed paths summarize two sequences of t decisions. Equal four-bit contributions at the k inspected coordinates leave the same function of every remaining assignment y . There are at most 16^k contribution tuples at each level.

An OBDD needs at most one node per distinct remaining function at each level.⁵ There are $N + 1$

⁵This is the Sieling–Wegener bound [64]; see also Bollig and Wegener [9, Theorem 1].

levels, including the terminal level after all bits have been fixed, giving at most

$$(N + 1)16^k \quad \text{nodes.} \quad \square$$

A small d-DNNF for L_n

✓ PositiveSide: small_dDNNF_size

Lemma 9. *For every choice of E , the corresponding function L_n has*

$$\text{d-DNNF-size}(L_n) = 2^{O(n)}. \quad (2.13)$$

Proof. Each outer term inspects at most $10n$ coordinates by Lemma 5, so Lemma 8 gives an OBDD with at most $(N + 1)16^{10n}$ nodes for each substituted term. An OBDD with s nodes has an equivalent d-DNNF with size $O(s)$ [24, Proposition 3.1]. Replace a decision on x_j by

$$(x_j \wedge C_1) \vee (\neg x_j \wedge C_0),$$

where C_0, C_1 are the circuits for its two continuations. Neither continuation tests x_j again, so the AND gates are decomposable. The OR gate is deterministic because its branches require opposite values of x_j . Shared continuations remain shared, and each decision adds only a constant number of edges. Thus each substituted term has a d-DNNF of size

$$O((N + 1)16^{10n}) = 2^{O(n)}.$$

Take the OR of these circuits. If two accepted the same assignment x , two distinct outer terms would both be true on $(Q((Ex)_u))_{u \in \Omega}$, contradicting Lemma 4. Hence the top OR is deterministic, and its children are d-DNNFs. The outer DNF in (2.7) has at most $n \cdot 2^n$ terms; including one edge from each term to the top OR gives size

$$1 + n \cdot 2^n \cdot 2^{O(n)} = 2^{O(n)}. \quad \square$$

3 Proving that zero squares are small

We now turn F_n into a two-party function by evaluating it on the sum of Alice's and Bob's inputs.⁶ Alice receives a full vector $\mathbf{a} = (a_u)_{u \in \Omega} \in G$, and Bob receives $\mathbf{b} = (b_u)_{u \in \Omega} \in G$. Their entries $a_u, b_u \in V$ are the four-bit inputs at coordinate u . Their task is to compute $F_n(\mathbf{a} + \mathbf{b})$. Since the sum has entry $a_u + b_u$ at coordinate u , their two-party function is

$$(\mathbf{a}, \mathbf{b}) \mapsto F_n(\mathbf{a} + \mathbf{b}) = f_n\left((Q(a_u + b_u))_{u \in \Omega}\right). \quad (3.1)$$

For $I \subseteq G$, call $I \times I$ a *zero square* if $F_n(\mathbf{a} + \mathbf{b}) = 0$ for every $\mathbf{a}, \mathbf{b} \in I$. We will show that such a set I occupies at most a $2^{-\Omega(n^2)}$ fraction of G .

3.1 A matrix supported on one-inputs

We prove this bound by assigning nonnegative weights to pairs $(\mathbf{a}, \mathbf{b})$ with $F_n(\mathbf{a} + \mathbf{b}) = 1$ and analyzing the resulting matrix K .⁷ Every entry within a zero square will then be zero; the task is to bound how large such a square block of zeros can be.

We need not assign positive weight to every pair with $F_n(\mathbf{a} + \mathbf{b}) = 1$. Leaving some of these entries zero can only create more zero squares. Thus, if we can rule out large zero squares in K , we also rule them out in the Boolean matrix with entries $F_n(\mathbf{a} + \mathbf{b})$.

⁶For comparison, Pabbaraju's construction uses a three-bit cyclic gadget over $\mathbb{Z}_8$ [58, Sec. 4]. Our construction instead uses the four-bit inner-product gadget Q , which allows the encoder to be linear over $\mathbb{F}_2$.

⁷The argument uses a weighted adjacency matrix and the weighted form of Hoffman's ratio bound on independent sets; see Haemers [34] for background. This matrix strategy adapts Pabbaraju's construction and eigenvalue argument [58, Sections 4.2–4.4]. We use different local weights and average over all label sets of size at least $n/3$, rather than those of one fixed size.

By (2.9), f_n accepts the assignment that is 1 exactly on $P_{i,S}$. We will assign weight to pairs whose gadget outputs give this assignment.

The two local weight functions

Lean: OuterMatrix

Definition 10. Define two nonnegative weight functions $\omega_0, \omega_1 : V \rightarrow \mathbb{R}_{\geq 0}$. For a four-bit string $z = (w, w') \in V$, their values are given by the following tables. The table rows and columns are the two halves w and w' of the single string z .

		$\omega_0(w, w')$						$\omega_1(w, w')$			
$w \backslash w'$		00	01	10	11	$w \backslash w'$		00	01	10	11
00		$\frac{15}{24}$	$\frac{1}{24}$	$\frac{1}{24}$	$\frac{1}{24}$	00		0	0	0	0
01		$\frac{1}{24}$	0	$\frac{1}{24}$	0	01		0	$\frac{4}{24}$	0	$\frac{4}{24}$
10		$\frac{1}{24}$	$\frac{1}{24}$	0	0	10		0	0	$\frac{4}{24}$	$\frac{4}{24}$
11		$\frac{1}{24}$	0	0	$\frac{1}{24}$	11		0	$\frac{4}{24}$	$\frac{4}{24}$	0

ω_0 is zero unless $Q(z) = 0$, while ω_1 is zero unless $Q(z) = 1$. The entries in each table sum to 1.

We now multiply these local weights to assign a weight to each pair of full vectors.

The product matrices and their average

Lean: OuterMatrix

Definition 11. For each bucket $i \in [n]_0$ and each label set $S \subseteq [n]_0$ satisfying $3 \cdot |S| \geq n$, define the weight of a pair of vectors $(\mathbf{a}, \mathbf{b})$ using one factor for each of the n^2 Boolean coordinates. At a coordinate $u \in P_{i,S}$ use $\omega_1(a_u + b_u)$; at every other coordinate use $\omega_0(a_u + b_u)$. Thus there are $|S|$ factors of ω_1 and $n^2 - |S|$ factors of ω_0 . Define the product matrix

$$K_{i,S}(\mathbf{a}, \mathbf{b}) = \prod_{u \in P_{i,S}} \omega_1(a_u + b_u) \prod_{u \notin P_{i,S}} \omega_0(a_u + b_u). \quad (3.2)$$

Finally, choose i uniformly from $[n]_0$ and, independently, choose S uniformly among the label sets satisfying $3 \cdot |S| \geq n$. Define K as their entrywise average:

$$K = \mathbb{E}_{i,S}[K_{i,S}].$$

Three elementary properties of K

✓ OuterMatrix: outerMatrix_properties

Lemma 12. K has the following properties:

- (a) K is real, symmetric, and entrywise nonnegative;
- (b) $K(\mathbf{a}, \mathbf{b}) > 0$ only if $F_n(\mathbf{a} + \mathbf{b}) = 1$;
- (c) every row of K sums to 1.

Proof. For property (a), every entry is real and nonnegative by the two tables. Swapping $\mathbf{a}$ and $\mathbf{b}$ leaves every sum $a_u + b_u$ unchanged, so all the product matrices, and hence their average K , are symmetric.

For property (b), suppose that $K(\mathbf{a}, \mathbf{b}) > 0$. Since K is an average of nonnegative product matrices, at least one of them has a positive entry at $(\mathbf{a}, \mathbf{b})$. Choose i and S with $K_{i,S}(\mathbf{a}, \mathbf{b}) > 0$. Every factor in (3.2) is nonnegative, so every factor must be positive. The supports shown in the two tables give, for every $u \in \Omega$,

$$\begin{aligned} u \in P_{i,S} &\implies \omega_1(a_u + b_u) > 0 \implies Q(a_u + b_u) = 1, \\ u \notin P_{i,S} &\implies \omega_0(a_u + b_u) > 0 \implies Q(a_u + b_u) = 0. \end{aligned}$$

The gadget outputs are therefore 1 exactly on $P_{i,S}$. Equation (2.9) says that f_n accepts this Boolean vector, so (3.1) gives $F_n(\mathbf{a} + \mathbf{b}) = 1$.

For property (c), fix a row $\mathbf{a}$ of one product matrix. As $\mathbf{b}$ ranges over G , the sum $\mathbf{a} + \mathbf{b}$ also ranges over all of G . Since $G = V^\Omega$, distributivity and Definition 10 give

$$\sum_{\mathbf{b} \in G} K_{i,S}(\mathbf{a}, \mathbf{b}) = \prod_{\mathbf{u} \in P_{i,S}} \left(\sum_{\mathbf{z} \in V} \omega_1(\mathbf{z}) \right) \prod_{\mathbf{u} \notin P_{i,S}} \left(\sum_{\mathbf{z} \in V} \omega_0(\mathbf{z}) \right) = 1.$$

Thus every product matrix has row sum 1, and so does their average K . $\square$

3.2 Parity vectors and their local inner products

To study the eigenvalues of K , we introduce parity vectors.

Parity vectors

Lean: ParityVectors

Definition 13. For a four-bit parameter $\xi = (\xi_1, \dots, \xi_4) \in V$, define the parity vector par_ξ by

$$\text{par}_\xi(z) = (-1)^{\xi \cdot z}, \quad \text{where} \quad \xi \cdot z = \xi_1 z_1 + \xi_2 z_2 + \xi_3 z_3 + \xi_4 z_4 \pmod{2}.$$

Here $z = (z_1, \dots, z_4)$ ranges over V . The sixteen choices of z give the sixteen entries of this vector. A full parameter $\xi = (\xi_u)_{u \in \Omega} \in G$ similarly determines a parity vector on G by

$$\text{par}_\xi(\mathbf{a}) = \prod_{u \in \Omega} \text{par}_{\xi_u}(a_u).$$

Its sign records the parity of the local inner products $\xi_u \cdot a_u$.

Parity identities

✓ ParityVectors: parity_identities

Lemma 14. For $\xi, z, z' \in V$ and $\xi, \xi' \in G$,

$$\text{par}_\xi(z + z') = \text{par}_\xi(z) \text{par}_\xi(z') \quad \text{and} \quad \text{par}_\xi(z) \text{par}_{\xi'}(z) = \text{par}_{\xi + \xi'}(z). \quad (3.3)$$

For every nonzero $\xi \in V$ and every nonzero $\xi \in G$,

$$\sum_{z \in V} \text{par}_\xi(z) = 0 \quad \text{and} \quad \sum_{z \in G} \text{par}_\xi(z) = 0. \quad (3.4)$$

Proof. The local identity in (3.3) follows from linearity of the dot product over $\mathbb{F}_2$:

$$\text{par}_\xi(z + z') = (-1)^{\xi \cdot (z + z')} = (-1)^{\xi \cdot z} (-1)^{\xi \cdot z'} = \text{par}_\xi(z) \text{par}_\xi(z').$$

For the global identity in (3.3), expand both global parity vectors using Definition 13, then combine their factors coordinate by coordinate:

$$\text{par}_\xi(z) \text{par}_{\xi'}(z) = \left(\prod_{u \in \Omega} \text{par}_{\xi_u}(z_u) \right) \left(\prod_{u \in \Omega} \text{par}_{\xi'_u}(z_u) \right) = \prod_{u \in \Omega} \text{par}_{\xi_u + \xi'_u}(z_u) = \text{par}_{\xi + \xi'}(z).$$

For the local identity in (3.4), choose a bit position k where ξ is 1. Pair each $z \in V$ with the vector z' obtained by flipping bit k . Flipping the same bit again returns z , so these vectors form disjoint pairs. Moreover,

$$\xi \cdot z' = \xi \cdot z + 1 \pmod{2},$$

so the two local parity signs in every pair are opposite and cancel.

The same argument proves the global identity in (3.4): choose a nonzero entry ξ_u of ξ , apply the local pairing at coordinate u , and hold every other coordinate fixed. $\square$

The weight functions ω_0, ω_1 and each local parity vector par_ξ are vectors indexed by V . We compute their inner products; the resulting numbers will become factors in the eigenvalues of K .⁸

One-coordinate weight-parity inner products

✓ LocalInnerProducts: weight_parity_inner_products

Lemma 15. For $r \in \{0, 1\}$ and $\xi \in V$, the inner product is

$$\langle \omega_r, \text{par}_\xi \rangle = \sum_{z \in V} \omega_r(z) \text{par}_\xi(z).$$

The pair of inner products is

$$(\langle \omega_0, \text{par}_\xi \rangle, \langle \omega_1, \text{par}_\xi \rangle) = \begin{cases} (1, 1), & \xi = (00, 00), \\ (\frac{2}{3}, -\frac{1}{3}), & \xi \neq (00, 00) \text{ and } Q(\xi) = 0, \\ (\frac{1}{2}, \frac{1}{3}), & \xi \neq (00, 00) \text{ and } Q(\xi) = 1. \end{cases}$$

In particular, every $\langle \omega_0, \text{par}_\xi \rangle$ is positive.

Proof. At $\xi = (00, 00)$, every parity entry is 1, so both inner products are 1 by Definition 10. Appendix B.2 proves the remaining cases. $\square$

Example Computing the two inner products for one parameter

At $\xi = (01, 00)$, the parity $\text{par}_\xi(w, w')$ is +1 in rows $w = 00, 10$ and -1 in rows $w = 01, 11$. Adding the positive rows and subtracting the negative rows gives

$$\langle \omega_0, \text{par}_{(01,00)} \rangle = \frac{18 + 2 - 2 - 2}{24} = \frac{2}{3}, \quad \langle \omega_1, \text{par}_{(01,00)} \rangle = \frac{0 + 8 - 8 - 8}{24} = -\frac{1}{3}.$$

3.3 Computing the eigenvalues of K

The entries of every product matrix $K_{i,S}$ depend only on the sum of their row and column indices. The following calculation identifies the corresponding eigenvectors.

Matrices determined by vector sums

✓ Eigenvalues: sum_matrix_eigenvectors

Lemma 16. Let $\omega : G \rightarrow \mathbb{R}$, and define the matrix M indexed by G by

$$M(\mathbf{a}, \mathbf{b}) = \omega(\mathbf{a} + \mathbf{b}), \quad \mathbf{a}, \mathbf{b} \in G.$$

Then every parity vector par_ξ is an eigenvector of M , with eigenvalue

$$\sum_{z \in G} \omega(z) \text{par}_\xi(z). \tag{3.5}$$

Proof. For $\mathbf{a}, \mathbf{z} \in G$, the local multiplication identity (3.3) extends to full inputs by the definition of the global parity vector:

$$\begin{aligned} \text{par}_\xi(\mathbf{a} + \mathbf{z}) &= \prod_{u \in \Omega} \text{par}_{\xi_u}(a_u + z_u) = \prod_{u \in \Omega} (\text{par}_{\xi_u}(a_u) \text{par}_{\xi_u}(z_u)) \\ &= \text{par}_\xi(\mathbf{a}) \text{par}_\xi(\mathbf{z}). \end{aligned} \tag{3.6}$$

Fix the row index $\mathbf{a}$. The substitution $\mathbf{z} = \mathbf{a} + \mathbf{b}$ is a bijection on G , with $\mathbf{b} = \mathbf{a} + \mathbf{z}$ because every vector

⁸These sums are the unnormalized Fourier coefficients (Walsh-Hadamard transforms) of ω_0 and ω_1 ; see O'Donnell [55, Section 1.3] for the parity-basis formulation.

is its own negative. Using (3.6) gives

$$\begin{aligned} (M \text{par}_\xi)(a) &= \sum_{b \in G} \omega(a+b) \text{par}_\xi(b) = \sum_{z \in G} \omega(z) \text{par}_\xi(a+z) \\ &= \text{par}_\xi(a) \sum_{z \in G} \omega(z) \text{par}_\xi(z). \end{aligned}$$

The last sum is independent of a , so it is the eigenvalue.⁹ □

We now combine the inner products from Lemma 15 across the product space $G = V^\Omega$.

A parity eigenbasis for K

✓ Eigenvalues: parity_eigenbasis

Lemma 17. For every $\xi \in G$, the parity vector par_ξ is an eigenvector of every product matrix $K_{i,S}$, with eigenvalue

$$\prod_{u \in P_{i,S}} \langle \omega_1, \text{par}_{\xi_u} \rangle \prod_{u \notin P_{i,S}} \langle \omega_0, \text{par}_{\xi_u} \rangle. \quad (3.7)$$

It is therefore also an eigenvector of K . These parity vectors form an orthogonal basis of $\mathbb{R}^G$.

Proof. Fix $\xi \in G$ and one product matrix $K_{i,S}$. Equation (3.2) shows that this matrix depends on a and b only through $a+b$. By Lemma 16, its eigenvalue on par_ξ is given by (3.5). For this product matrix, we obtain

$$\begin{aligned} &\sum_{z \in G} \prod_{u \in P_{i,S}} \omega_1(z_u) \prod_{u \notin P_{i,S}} \omega_0(z_u) \text{par}_\xi(z) \\ &= \sum_{z \in G} \prod_{u \in P_{i,S}} (\omega_1(z_u) \text{par}_{\xi_u}(z_u)) \prod_{u \notin P_{i,S}} (\omega_0(z_u) \text{par}_{\xi_u}(z_u)) \\ &= \prod_{u \in P_{i,S}} \left(\sum_{z \in V} \omega_1(z) \text{par}_{\xi_u}(z) \right) \prod_{u \notin P_{i,S}} \left(\sum_{z \in V} \omega_0(z) \text{par}_{\xi_u}(z) \right) \\ &= \prod_{u \in P_{i,S}} \langle \omega_1, \text{par}_{\xi_u} \rangle \prod_{u \notin P_{i,S}} \langle \omega_0, \text{par}_{\xi_u} \rangle. \end{aligned}$$

This is (3.7). Since K is the average of the product matrices, the same parity vector is an eigenvector of K , with eigenvalue equal to the average of their eigenvalues.

Now fix distinct $\xi, \xi' \in G = V^\Omega$. Their coordinatewise sum $\xi + \xi'$ is nonzero. Equations (3.3) and (3.4) therefore show that the parity vectors are orthogonal:

$$\langle \text{par}_\xi, \text{par}_{\xi'} \rangle = \sum_{z \in G} \text{par}_\xi(z) \text{par}_{\xi'}(z) = \sum_{z \in G} \text{par}_{\xi+\xi'}(z) = 0.$$

As ξ ranges over G , we obtain $|G|$ nonzero, mutually orthogonal vectors in the $|G|$ -dimensional space $\mathbb{R}^G$. They therefore form a basis. □

The eigenvalues of K

✓ LeastEigenvalue: eigenvalue_formula

Lemma 18. We express each eigenvalue as a positive factor times an average over buckets. For a global parameter $\xi = (\xi_u)_{u \in \Omega} \in G$, define the positive factor

$$\gamma(\xi) = \prod_{u \in \Omega} \langle \omega_0, \text{par}_{\xi_u} \rangle.$$

⁹These parity vectors are the Fourier characters of the additive group. Lemma 16 is the standard diagonalization of a convolution operator, or weighted Cayley matrix, by characters; see O’Donnell [55, Section 1.5].

Let $\mathbf{S}$ be a uniformly random subset of $[n]_0$. For each bucket $i \in [n]_0$, define the average

$$\mu_i(\xi) := \mathbb{E}_{\mathbf{S}} \left[\prod_{u \in P_{i,\mathbf{S}}} q_{\xi_u} \mid 3 \cdot |\mathbf{S}| \geq n \right], \quad \text{where} \quad q_{\xi} := \frac{\langle \omega_1, \text{par}_{\xi} \rangle}{\langle \omega_0, \text{par}_{\xi} \rangle}.$$

Then the eigenvalue λ_{ξ} of K corresponding to par_{ξ} is

$$\lambda_{\xi} = \frac{\gamma(\xi)}{n} \sum_{i \in [n]_0} \mu_i(\xi). \quad (3.8)$$

Proof. By Lemma 17, the eigenvalue of the product matrix $K_{i,S}$ on par_{ξ} factors as

$$\prod_{u \in P_{i,S}} \langle \omega_1, \text{par}_{\xi_u} \rangle \prod_{u \notin P_{i,S}} \langle \omega_0, \text{par}_{\xi_u} \rangle = \gamma(\xi) \prod_{u \in P_{i,S}} q_{\xi_u}.$$

Averaging over the eligible label sets replaces the last product by $\mu_i(\xi)$. Averaging over $i \in [n]_0$ then gives (3.8), since K is the average of the product matrices. Since the parity eigenvectors form a basis, these are all the eigenvalues of K . $\square$

3.4 Bounding the least eigenvalue

Equation (3.8) separates the sign from the magnitude. Since $\gamma(\xi) > 0$, the sign of λ_{ξ} is determined by the average of the $\mu_i(\xi)$. We first prove that if ξ has few nonzero coordinates, this average and hence λ_{ξ} are nonnegative. When there are many nonzero coordinates, they instead make $\gamma(\xi)$ exponentially small.

The condition inside μ_i discards the label sets with fewer than $n/3$ labels. We first record that such sets are rare.

Most label sets are eligible

✓ LabelSets: most_label_sets_eligible

Lemma 19. For uniformly random $\mathbf{S} \subseteq [n]_0$,

$$\Pr_{\mathbf{S}}[3 \cdot |\mathbf{S}| < n] \leq 2^{-n/15}.$$

Proof. Every excluded set $\mathbf{S}$ satisfies $2^{n/3-|\mathbf{S}|} \geq 1$, so the number of excluded sets is at most the sum of these nonnegative weights over all label sets:

$$\Pr_{\mathbf{S}}[3 \cdot |\mathbf{S}| < n] = 2^{-n} |\{S \subseteq [n]_0 : 3 \cdot |S| < n\}| \leq 2^{-n} \sum_{S \subseteq [n]_0} 2^{n/3-|S|}.$$

Grouping by size gives the binomial sum; the binomial theorem and $(3/2)^5 < 2^3$ finish the estimate:

$$\Pr_{\mathbf{S}}[3 \cdot |\mathbf{S}| < n] \leq 2^{-2n/3} \sum_{k=0}^n \binom{n}{k} 2^{-k} = 2^{-2n/3} \left(1 + \frac{1}{2}\right)^n = 2^{-2n/3} \left(\frac{3}{2}\right)^n \leq 2^{-2n/3} \cdot 2^{3n/5} = 2^{-n/15}.$$

$\square$

Negative eigenvalues need many nonzeros

✓ LeastEigenvalue: negative_needs_many

Lemma 20. For $\xi \in G$, if λ_{ξ} is negative, then more than $n^2/30$ coordinates of ξ must be nonzero:

$$\lambda_{\xi} < 0 \quad \implies \quad |\{u \in \Omega : \xi_u \neq (00, 00)\}| > \frac{n^2}{30}.$$

Intuition. The eigenvalue averages products over label sets containing at least $n/3$ labels. Temporarily including all label sets gives a positive average. Nonzero coordinates can make this positive average smaller. The proof shows that, with few nonzero coordinates overall, the average across buckets remains large enough that removing the extra label sets cannot make it negative.

Proof. We prove the contrapositive. Fix $\xi \in G$ with at most $n^2/30$ nonzero coordinates; we prove $\lambda_\xi \geq 0$. Since $\gamma(\xi) > 0$, Equation (3.8) shows that it is enough to prove

$$\sum_{i \in [n]_0} \mu_i(\xi) \geq 0.$$

Fix $i \in [n]_0$. To bound $\mu_i(\xi)$, we first sum over all label sets, without the size restriction; we will remove the ineligible sets afterward.

Example Summing over the four subsets of two labels

With labels 0 and 1, the four subsets contribute the following products (the empty product is 1):

$$\underbrace{1}_{S=\emptyset} + \underbrace{q_{\xi(i,0)}}_{S=\{0\}} + \underbrace{q_{\xi(i,1)}}_{S=\{1\}} + \underbrace{q_{\xi(i,0)}q_{\xi(i,1)}}_{S=\{0,1\}} = (1 + q_{\xi(i,0)})(1 + q_{\xi(i,1)}).$$

For n labels, expanding the product of the factors $(1 + q_{\xi(i,a)})$ likewise gives one product for each subset S : choose $q_{\xi(i,a)}$ when $a \in S$ and 1 otherwise. Thus

$$\sum_{S \subseteq [n]_0} \prod_{u \in P_{i,S}} q_{\xi_u} = \prod_{u \in B_i} (1 + q_{\xi_u}).$$

To bound this product from below, write ℓ_i for the number of nonzero coordinates in B_i :

$$\ell_i = |\{u \in B_i : \xi_u \neq (00, 00)\}|.$$

If $\xi_u = (00, 00)$, then $q_{\xi_u} = 1$. At every other coordinate, Lemma 15 gives $q_{\xi_u} \geq -1/2$. The $n - \ell_i$ zero coordinates therefore contribute factors of 2, and the other ℓ_i factors are at least $1/2$:

$$\prod_{u \in B_i} (1 + q_{\xi_u}) \geq 2^{n-\ell_i} \left(\frac{1}{2}\right)^{\ell_i} = 2^n 4^{-\ell_i}.$$

We now remove the label sets with $3 \cdot |S| < n$. Write $p := \Pr_{\mathbf{S}}[3 \cdot |\mathbf{S}| < n]$ for the probability that a uniformly chosen label set is excluded. There are $2^n p$ such sets, and Lemma 15 gives $|q_\xi| \leq 1$. Each removed product is therefore at most 1, so removing all of them decreases the sum by at most $2^n p$:

$$\sum_{\substack{S \subseteq [n]_0 \\ 3 \cdot |S| \geq n}} \prod_{u \in P_{i,S}} q_{\xi_u} \geq 2^n 4^{-\ell_i} - 2^n p.$$

The average $\mu_i(\xi)$ divides this retained sum by the number $2^n(1-p)$ of eligible label sets. Hence

$$\mu_i(\xi) \geq \frac{2^n 4^{-\ell_i} - 2^n p}{2^n(1-p)} = \frac{4^{-\ell_i} - p}{1-p}. \quad (3.9)$$

Sum (3.9) over the n buckets:

$$\sum_{i \in [n]_0} \mu_i(\xi) \geq \frac{1}{1-p} \left(\sum_{i \in [n]_0} 4^{-\ell_i} - np \right).$$

It is therefore enough to show that the sum of the $4^{-\ell_i}$ is at least np . The arithmetic-geometric mean inequality says that the arithmetic mean of the n positive numbers $4^{-\ell_i}$ is at least their geometric mean:

$$\sum_{i \in [n]_0} 4^{-\ell_i} \geq n \left(\prod_{i \in [n]_0} 4^{-\ell_i} \right)^{1/n} = n 4^{-\frac{1}{n} \sum_{i \in [n]_0} \ell_i}.$$

The hypothesis on ξ gives $\frac{1}{n} \sum_{i \in [n]_0} \ell_i \leq n/30$, and Lemma 19 gives $p \leq 2^{-n/15}$. Substituting the bounds on $\frac{1}{n} \sum_{i \in [n]_0} \ell_i$ and p gives

$$\sum_{i \in [n]_0} 4^{-\ell_i} \geq n 4^{-n/30} = n 2^{-n/15} \geq np.$$

Thus $\sum_{i \in [n]_0} \mu_i(\xi) \geq 0$. Since $\gamma(\xi) > 0$, Equation (3.8) now gives $\lambda_\xi \geq 0$. $\square$

A lower bound on every eigenvalue of K

✓ LeastEigenvalue: spectral_estimate

Lemma 21. *The least eigenvalue of K satisfies*

$$\lambda_{\min}(K) \geq -\varepsilon, \quad \text{where} \quad \varepsilon := \left(\frac{2}{3}\right)^{n^2/30} = 2^{-\Omega(n^2)}. \quad (3.10)$$

Proof. Fix $\xi \in G$. If $\lambda_\xi \geq 0$, the claimed bound is immediate. Otherwise, Lemma 20 shows that ξ has more than $n^2/30$ nonzero coordinates. Lemma 15 shows that, in $\gamma(\xi)$, a zero coordinate contributes 1, while every nonzero coordinate contributes a positive factor at most $2/3$. Hence

$$0 < \gamma(\xi) = \prod_{u \in \Omega} \langle \omega_0, \text{par}_{\xi_u} \rangle \leq \left(\frac{2}{3}\right)^{|\{u \in \Omega: \xi_u \neq (00,00)\}|} < \left(\frac{2}{3}\right)^{n^2/30}.$$

Each $\mu_i(\xi)$ is an average of products whose factors have absolute value at most 1. Hence $|\mu_i(\xi)| \leq 1$, so their average is at least $-\gamma(\xi)$. Equation (3.8) therefore gives

$$\lambda_\xi = \gamma(\xi) \frac{1}{n} \sum_{i \in [n]_0} \mu_i(\xi) \geq -\gamma(\xi) \geq -\left(\frac{2}{3}\right)^{n^2/30}.$$

$\square$

Why we use Q . Different four-bit inputs can have the same output under Q , so we can distribute weight among them without changing the bit seen by f_n . Our weights ensure that a negative eigenvalue requires many nonzero coordinates in its parity parameter. Each such coordinate contributes a factor of at most $2/3$ to $\gamma(\xi)$, making the negative eigenvalue exponentially small in magnitude.

3.5 Zero squares are small

Zero squares are small

✓ ZeroSquares: small_zero_squares

Lemma 22. *If $I \subseteq G$ satisfies $F_n(a + b) = 0$ for all $a, b \in I$, then*

$$|I| \leq \varepsilon |G|. \quad (3.11)$$

Intuition. Every zero square of $F_n(a + b)$ must also be a zero square of the matrix K constructed above. We now use the properties of K to bound the size of any such square. The proof below will make the following comparison precise. Every row of K has total weight 1. If that weight were spread uniformly

across the columns, the square $I \times I$ would contain total weight $|I|^2/|G|$. The matrix K need not spread its weight uniformly. But the bound on its negative eigenvalues guarantees that the total weight inside the square is at least $|I|^2/|G| - \varepsilon|I|$. If $|I| > \varepsilon|G|$, this lower bound is positive, so the entries in the square cannot all be zero.

Proof. Let $\mathbf{1}_I$ be the vector that is 1 on I and 0 elsewhere. The zero-square assumption and properties (a) and (b) of Lemma 12 give $K(\mathbf{a}, \mathbf{b}) = 0$ whenever $\mathbf{a}, \mathbf{b} \in I$. Thus

$$\mathbf{1}_I^\top K \mathbf{1}_I = \sum_{\mathbf{a} \in I} \sum_{\mathbf{b} \in I} K(\mathbf{a}, \mathbf{b}) = 0.$$

We express this same vector in the parity eigenbasis from Lemma 17.¹⁰ The all-zero parameter gives $\text{par}_0(\mathbf{z}) = 1$ for every $\mathbf{z} \in G$, so the constant vector is among these parity vectors. Each parity vector has $|G|$ entries equal to $+1$ or -1 , so its squared length is $|G|$. Dividing each vector by $\sqrt{|G|}$ gives length 1. List the resulting vectors as $e_0, \dots, e_{|G|-1}$, with the constant vector $e_0 = \mathbf{1}/\sqrt{|G|}$ first. They remain mutually orthogonal. Write

$$\mathbf{1}_I = \sum_{j=0}^{|G|-1} c_j e_j, \quad K e_j = \lambda_j e_j.$$

The other basis vectors have positive and negative entries, so their contributions can cancel $c_0 e_0$ outside I . Every row of K sums to 1, so for each $\mathbf{a} \in G$,

$$(K e_0)(\mathbf{a}) = \sum_{\mathbf{b} \in G} K(\mathbf{a}, \mathbf{b}) \frac{1}{\sqrt{|G|}} = \frac{1}{\sqrt{|G|}} = e_0(\mathbf{a}).$$

Thus $\lambda_0 = 1$. We first compute $K \mathbf{1}_I$, using $K e_k = \lambda_k e_k$ on each basis vector:

$$K \mathbf{1}_I = K \left(\sum_{k=0}^{|G|-1} c_k e_k \right) = \sum_{k=0}^{|G|-1} c_k (K e_k) = \sum_{k=0}^{|G|-1} \lambda_k c_k e_k.$$

Now take the inner product with $\mathbf{1}_I$. Orthogonality removes the terms with $j \neq k$, and $\lambda_0 = 1$ gives the final equality:

$$\begin{aligned} 0 &= \mathbf{1}_I^\top K \mathbf{1}_I = \langle \mathbf{1}_I, K \mathbf{1}_I \rangle = \left\langle \sum_{j=0}^{|G|-1} c_j e_j, \sum_{k=0}^{|G|-1} \lambda_k c_k e_k \right\rangle \\ &= \sum_{j,k=0}^{|G|-1} c_j \lambda_k c_k \langle e_j, e_k \rangle = \sum_{j=0}^{|G|-1} \lambda_j c_j^2 = c_0^2 + \sum_{j=1}^{|G|-1} \lambda_j c_j^2. \end{aligned} \tag{3.12}$$

We now compute c_0 and $\sum_{j=0}^{|G|-1} c_j^2$ in terms of $|I|$. To find c_0 , take the inner product with e_0 . Since $\langle e_0, e_0 \rangle = 1$ and $\langle e_j, e_0 \rangle = 0$ for $j \neq 0$, this leaves only c_0 . Each entry of e_0 is $1/\sqrt{|G|}$, and the indicator selects the entries in I . Hence

$$c_0 = \sum_{j=0}^{|G|-1} c_j \langle e_j, e_0 \rangle = \langle \mathbf{1}_I, e_0 \rangle = \sum_{\mathbf{z} \in I} \frac{1}{\sqrt{|G|}} = \frac{|I|}{\sqrt{|G|}}.$$

Similarly, orthogonality lets us compute the sum of the squared coefficients:

$$\sum_{j=0}^{|G|-1} c_j^2 = \sum_{j,k=0}^{|G|-1} c_j c_k \langle e_j, e_k \rangle = \left\langle \sum_{j=0}^{|G|-1} c_j e_j, \sum_{k=0}^{|G|-1} c_k e_k \right\rangle = \|\mathbf{1}_I\|_2^2 = \sum_{\mathbf{z} \in G} \mathbf{1}_I(\mathbf{z})^2 = |I|.$$

¹⁰This is a form of Hoffman's ratio bound for a weighted matrix with common row sum 1 [34].

Lemma 21 gives $\lambda_j \geq -\varepsilon$ for every $j \geq 1$. Using this bound and the two coefficient calculations in (3.12) gives

$$0 = c_0^2 + \sum_{j=1}^{|G|-1} \lambda_j c_j^2 \geq \frac{|I|^2}{|G|} - \varepsilon \sum_{j=1}^{|G|-1} c_j^2 \geq \frac{|I|^2}{|G|} - \varepsilon |I|. \quad (3.13)$$

If I is empty, the claimed bound is immediate. Otherwise, rearranging (3.13) and dividing by $|I|$ gives

$$\frac{|I|^2}{|G|} \leq \varepsilon |I|, \quad \text{hence} \quad |I| \leq \varepsilon |G|.$$

□

4 Proving the DNNF lower bound

We now choose the linear encoder E and prove the DNNF lower bound for the corresponding function $\neg L_n$.

4.1 Choosing the encoder for the lower bound

We now choose the linear encoder matrix E from Definition 7 so that varying any set of at least $N/3$ input positions produces at least $|G|/8$ different encoded vectors.

Restricted encoders and their images

Lean: `RestrictedEncoder`

Definition 23. Let E be a linear encoder matrix as in Definition 7. For $J \subseteq [N]$, let E_J be the submatrix retaining the columns indexed by J . Its *image* is the set of vectors it can produce:

$$\text{im}(E_J) = \{E_J y : y \in \{0, 1\}^J\}.$$

The image contains zero and is closed under addition: for any two assignments,

$$E_J y + E_J y' = E_J (y + y') \in \text{im}(E_J).$$

The following observation applies to any set with this closure property.

Unchanged or doubled output sets

✓ EncoderIndex: `adjoining_vector_size`

Lemma 24. Let $B \subseteq G$ be closed under addition, and let $\mathbf{g} \in G$. Allowing the vectors in B to be used either with or without $\mathbf{g}$ gives the set

$$B' = B \cup \{\mathbf{b} + \mathbf{g} : \mathbf{b} \in B\}, \quad |B'| = \begin{cases} |B|, & \mathbf{g} \in B, \\ 2|B|, & \mathbf{g} \notin B. \end{cases}$$

Proof. If $\mathbf{g} \in B$, closure under addition puts every $\mathbf{b} + \mathbf{g}$ back in B , so $B' = B$. Otherwise the two sets in the union are disjoint. Indeed, an overlap would give $\mathbf{b} = \mathbf{b}' + \mathbf{g}$ for some $\mathbf{b}, \mathbf{b}' \in B$, and hence

$$\mathbf{g} = \mathbf{b} + \mathbf{b}' \in B,$$

a contradiction. Distinct $\mathbf{b}$ give distinct $\mathbf{b} + \mathbf{g}$, so the second set has $|B|$ elements as well. □

One encoder for every large set of positions

✓ EncoderExistence: encoder_with_large_images

Lemma 25. *There is a single choice of the encoding matrix E for which every submatrix E_J with $J \subseteq [N]$ and $|J| \geq N/3$ has a large image:*

$$|\text{im}(E_J)| \geq \frac{|G|}{8}.$$

We will choose E at random and show that the probability of any failure is less than one. If E_J has too small an image, all columns indexed by J lie in that small set, since each column is itself an output of E_J . Our strategy is to count the sets that could contain those columns, then show that a random matrix is unlikely to put so many columns in any one of them.

A containing set for each small image. We will count small images by enclosing them in sets of one common size. Start with the empty list of columns, whose only sum is zero, and process the columns of E_J one at a time. Lemma 24 says that the number of outputs either stays unchanged or doubles at each step. The final set is $\text{im}(E_J)$, so its size is a power of two. Consequently,

$$|\text{im}(E_J)| < \frac{|G|}{8} = 2^{4n^2-3} \implies |\text{im}(E_J)| \leq 2^{4n^2-4} = \frac{|G|}{16}.$$

If the image is smaller still, append a vector outside the current set of outputs to the list. This doubles the output set. Repeat until it has exactly $|G|/16$ elements. For each fixed pair (E, J) with a small image, choose one such containing set and denote it by $A_{E,J}$:

$$\text{im}(E_J) \subseteq A_{E,J}, \quad |A_{E,J}| = |G|/16.$$

The added vectors define only this containing set, not a change to E . Like every set of outputs of a binary matrix, $A_{E,J}$ is closed under addition.

We next count all addition-closed sets of this size.

Counting possible containing sets

✓ GeneratingLists: containing_sets_bound

Lemma 26. *There are at most 32^{4n^2-4} sets $A \subseteq G$ of size $|G|/16$ that are closed under addition.*

Proof. Set $r = 4n^2 - 4$, and fix one addition-closed set A of size $2^r = |G|/16$. A *generating list* for A is a list of vectors whose entries can be selected and added to obtain exactly the elements of A ; selecting no entries gives zero. We will count generating lists of length r .¹¹

First construct such a list. Start with the empty list, whose only sum is zero. This belongs to A because $\mathbf{a} + \mathbf{a} = \mathbf{0}$ for any $\mathbf{a} \in A$. At each step, choose a vector of A that is not a sum of entries already in the list. All obtainable sums stay in A by closure under addition, and each new choice doubles their number by Lemma 24. After $j < r$ choices there are only $2^j < |A|$ obtainable sums, so another choice is possible. After r choices there are $2^r = |A|$ sums, all in A , so the list generates exactly A .

Now count the lists produced by this construction. After $j < r$ choices, any vector of A not yet obtainable can be chosen next. The number of choices for the next entry is

$$|A| - 2^j \geq |A|/2.$$

Over the r steps, this gives

$$\#\{\text{lists generating } A\} \geq (|A|/2)^r = (|G|/32)^r.$$

Now sum over all possible containing sets A . No list generates two different sets, and there are $|G|^r$ ordered lists of length r in total, so

$$\sum_A \#\{\text{lists generating } A\} = \#\{\text{lists generating a containing set}\} \leq |G|^r.$$

¹¹These lists are ordered bases of binary vector subspaces. The counting argument is the one that gives the Gaussian binomial coefficient; see Stanley [65, Proposition 1.7.2]. Here we only need a coarse bound.

Each set contributes at least $(|G|/32)^r$ lists, so

$$\#\{\text{possible containing sets } A\} \leq \frac{|G|^r}{(|G|/32)^r} = 32^r.$$

□

Proof of Lemma 25. Choose each entry of E independently and uniformly from $\{0, 1\}$. Its columns $\mathbf{g}_1, \dots, \mathbf{g}_N$ are therefore independent uniform vectors in G .¹²

If $|\text{im}(E_J)| < |G|/8$, its containing set $A_{E,J}$ contains every column indexed by J . Thus a failure of the image bound forces all these columns into one of the sets counted in Lemma 26.

The probability that selected columns fit in a containing set. For the probability calculation, fix a pair consisting of a set J with $|J| \geq \frac{N}{3}$ and an addition-closed set A of size $|G|/16$, independently of the random matrix E . For this fixed pair, each random column belongs to A with probability $|A|/|G| = 1/16$. Independence of the columns gives

$$\Pr_E[\mathbf{g}_j \in A \text{ for every } j \in J] = 16^{-|J|} \leq 16^{-N/3}.$$

Keep J fixed, but now consider every possible containing set A . If E fails for this J , all columns indexed by J lie in a containing set $A_{E,J}$. Sum over all containing sets A , of which there are at most 32^r by Lemma 26, where $r = 4n^2 - 4$:

$$\Pr_E\left[|\text{im}(E_J)| < \frac{|G|}{8}\right] \leq \sum_A \Pr_E[\mathbf{g}_j \in A \text{ for every } j \in J] \leq 32^r \cdot 16^{-N/3}.$$

Finally, sum over all J with $|J| \geq \frac{N}{3}$. There are at most 2^N choices, so the union bound gives

$$\begin{aligned} \Pr_E[\text{failure for at least one } J] &\leq 2^N \cdot 32^r \cdot 16^{-N/3} \\ &= 2^{60n^2} \cdot 2^{5(4n^2-4)} \cdot 2^{-80n^2} = 2^{-20} < 1. \end{aligned}$$

Thus some encoding matrix E has $|\text{im}(E_J)| \geq \frac{|G|}{8}$ for every J with $|J| \geq \frac{N}{3}$. This proves Lemma 25. □

For the rest of the proof, fix an encoding matrix E supplied by Lemma 25, and let L_n be the corresponding function from Definition 7. Lemma 9 already gives its small d-DNNF.

4.2 From zero squares to a rectangle lower bound

Section 3 bounded zero squares for F_n . We now use the properties of the linear encoder E to turn this into a lower bound on the number of balanced zero-rectangles needed to cover the zero-inputs of $L_n(x) = F_n(Ex)$.

A rectangle allows assignments on two disjoint sets of input positions to be combined independently.

Balanced zero-rectangles

Lean: Rectangles

Definition 27. For $J \subseteq [N]$, let $J^c = [N] \setminus J$. A *rectangle* for the partition $[N] = J \dot{\cup} J^c$ is a set

$$R = \mathcal{A} \times \mathcal{B}, \quad \mathcal{A} \subseteq \{0, 1\}^J, \quad \mathcal{B} \subseteq \{0, 1\}^{J^c}.$$

A rectangle is a *zero-rectangle* for L_n when every assignment in it is a zero-input of L_n , equivalently when $R \subseteq L_n^{-1}(0)$. It is *balanced* when both sides contain at least one third of the Boolean positions:

$$\frac{N}{3} \leq |J| \leq \frac{2N}{3}. \quad (4.1)$$

¹²This argument is related to list decoding from erasures for binary linear codes; see Guruswami [33, Section 2.3] for background.

A small DNNF gives a small rectangle cover, by the theorem of Bova, Capelli, Mengel, and Slivovsky [10, Theorem 6], allowing here for gates with arbitrarily many inputs. The rectangles in a cover may use different partitions J, J^c .

A DNNF yields a small rectangle cover

✓ RectangleReduction: balanced_rectangle_cover

Lemma 28. *If a DNNF of size s computes $\neg L_n$, then $L_n^{-1}(0)$ is the union of $O(s)$ balanced zero-rectangles for L_n .*

Proof outline. A satisfying assignment has a *certificate*: start at the output, keep one satisfied input at each OR gate, and keep both inputs at each AND gate of a binary DNNF. Fix a gate v . Decomposability lets us combine the part below v from one certificate with the part outside v from another certificate containing v : the two parts use disjoint sets of variables. Thus the assignments with a certificate containing v form a rectangle. Using this observation, Bova, Capelli, Mengel, and Slivovsky [10] construct a cover with at most one rectangle per gate of the binary circuit, balancing each partition over the variables the circuit mentions. Each side therefore has at most $2N/3$ positions. Distribute the unused input positions between the two sides so that each has between $N/3$ and $2N/3$ positions, allowing both values at every added position. This preserves each rectangle and extends the cover to assignments on all N input positions. Appendix B.3 gives more details, including conversion to a binary DNNF with $O(s)$ nodes.

We count assignments by the encoded vectors they produce.

Fibers of a restricted encoder

Lean: RestrictedEncoder

Definition 29. For $J \subseteq [N]$ and $z \in \text{im}(E_J)$, the *fiber* over z is the set of assignments producing that sum:

$$E_J^{-1}(z) = \{y \in \{0, 1\}^J : E_J y = z\}.$$

We will show that a balanced zero-rectangle for L_n contains only a small fraction of the zero fiber $E^{-1}(0)$. These assignments are zero-inputs of L_n , because

$$L_n(x) = F_n(Ex) = F_n(0) = 0 \quad (x \in E^{-1}(0))$$

by (2.12). Thus a cover of all zero-inputs must also cover $E^{-1}(0)$.

Equal-sized fibers

✓ FiberCounts: equal_fiber_sizes

Lemma 30. *For any choice of encoding vectors, the maps E_J have equally sized fibers:*

$$|E_J^{-1}(z)| = \frac{2^{|J|}}{|\text{im}(E_J)|} \quad \text{for all } J \subseteq [N] \text{ and } z \in \text{im}(E_J).$$

Proof. Fix $J \subseteq [N]$ and any $z \in \text{im}(E_J)$. Choose $x \in E_J^{-1}(z)$. We construct a bijection $E_J^{-1}(z) \longleftrightarrow E_J^{-1}(0)$ by adding this fixed x in either direction. For x' in the fiber over z , the assignment $x' + x$ belongs to the fiber over zero:

$$E_J(x' + x) = E_J x' + E_J x = z + z = 0.$$

The inverse sends y in the fiber over zero to $y + x$ in the fiber over z :

$$E_J(y + x) = E_J y + E_J x = 0 + z = z.$$

These maps undo each other because, for any assignment $t \in \{0, 1\}^J$,

$$(t + x) + x = t + (x + x) = t.$$

This proves that the map is a bijection, so the two fibers have the same size:

$$|E_J^{-1}(z)| = |E_J^{-1}(0)|.$$

Since $\mathbf{z}$ was arbitrary in $\text{im}(E_J)$, every fiber has the same size as the fiber over zero.

The $|\text{im}(E_J)|$ fibers partition the $2^{|J|}$ assignments into equally sized sets, giving

$$|E_J^{-1}(\mathbf{z})| = \frac{2^{|J|}}{|\text{im}(E_J)|}.$$

□

We now combine these fiber counts with the zero-square bound.

Balanced zero-rectangle estimate

✓ ZeroFiber: rectangle_estimate

Lemma 31. *With $\varepsilon = (2/3)^{n^2/30}$ as in Lemma 21, every balanced zero-rectangle R for L_n contains at most a 64ε fraction of $E^{-1}(\mathbf{0})$:*

$$\frac{|R \cap E^{-1}(\mathbf{0})|}{|E^{-1}(\mathbf{0})|} \leq 64\varepsilon. \quad (4.2)$$

Proof. Fix a balanced zero-rectangle $R = \mathcal{A} \times \mathcal{B}$ for the partition $[N] = J \dot{\cup} J^c$. For $x = (x_J, x_{J^c})$ in this rectangle, the two sides contribute separately to the encoded vector:

$$Ex = E_J x_J + E_{J^c} x_{J^c}.$$

Their sum is zero exactly when the two contributions are equal. We group the assignments in $R \cap E^{-1}(\mathbf{0})$ according to the vector produced by both parts. We will count the assignments in each group and then add these counts.

Few vectors can be produced on both sides. Define the sets of encoded vectors available on the two sides:

$$A^* = \{E_J x_J : x_J \in \mathcal{A}\}, \quad B^* = \{E_{J^c} x_{J^c} : x_{J^c} \in \mathcal{B}\}.$$

Choose any $\mathbf{a} \in A^*$ and $\mathbf{b} \in B^*$. By their definitions, there are assignments $x_J \in \mathcal{A}$ and $x_{J^c} \in \mathcal{B}$ with

$$E_J x_J = \mathbf{a}, \quad E_{J^c} x_{J^c} = \mathbf{b}.$$

Since $R = \mathcal{A} \times \mathcal{B}$, combining these assignments gives $x = (x_J, x_{J^c}) \in R$. Its encoded vector is

$$Ex = E_J x_J + E_{J^c} x_{J^c} = \mathbf{a} + \mathbf{b}.$$

The rectangle R contains only zero-inputs of L_n , so

$$0 = L_n(x) = F_n(Ex) = F_n(\mathbf{a} + \mathbf{b}). \quad (4.3)$$

Now let $I = A^* \cap B^*$ be the set of encoded vectors that can be produced both by an assignment in $\mathcal{A}$ and by one in $\mathcal{B}$. Any $\mathbf{a}, \mathbf{b} \in I$ satisfy (4.3), so $I \times I$ is a zero square for $(\mathbf{a}, \mathbf{b}) \mapsto F_n(\mathbf{a} + \mathbf{b})$. Lemma 22 therefore gives

$$|I| \leq \varepsilon |G|. \quad (4.4)$$

Figure 3 illustrates the passage from assignments in R to pairs of encoded vectors.

Count assignments whose two encoder outputs agree. For each $\mathbf{z} \in I$, count the pairs $x_J \in \mathcal{A}$ and $x_{J^c} \in \mathcal{B}$ for which

$$E_J x_J = E_{J^c} x_{J^c} = \mathbf{z}.$$

As $\mathbf{z}$ ranges over I , these pairs give exactly the assignments in $R \cap E^{-1}(\mathbf{0})$. Each such assignment determines $\mathbf{z}$ uniquely, so summing over $\mathbf{z} \in I$ counts each assignment once:

$$|R \cap E^{-1}(\mathbf{0})| = \sum_{\mathbf{z} \in I} |\mathcal{A} \cap E_J^{-1}(\mathbf{z})| \cdot |\mathcal{B} \cap E_{J^c}^{-1}(\mathbf{z})|.$$

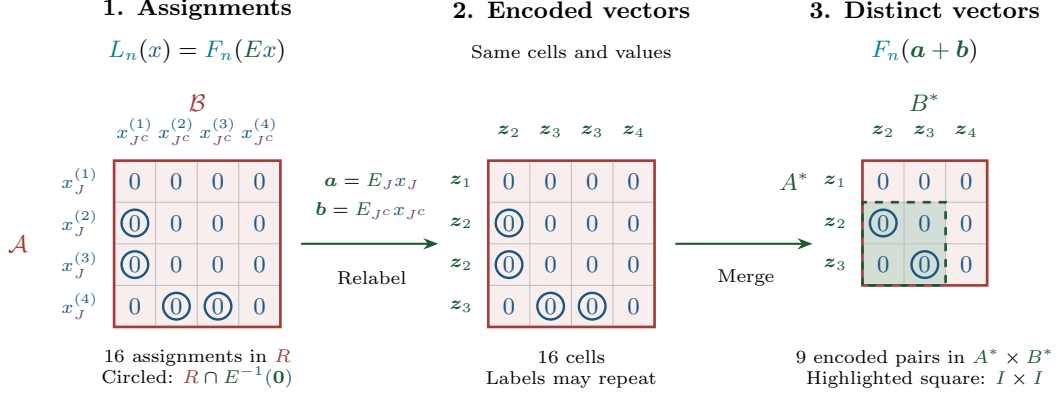


Figure 3: From assignments to encoded pairs (schematic). Encoding changes the labels, not the cell values; $z_1, \dots, z_4$ are distinct vectors. In the middle grid, rows 2 and 3 have equal labels, as do columns 2 and 3. Merging them leaves nine encoded pairs in $A^* \times B^*$. The four circled assignments map to (z_2, z_2) and (z_3, z_3) ; the highlighted square uses $I = \{z_2, z_3\}$ on both axes.

Each intersection is contained in a full fiber. Both sides of the balanced partition have at least $N/3$ positions, so Lemmas 30 and 25 give

$$|\mathcal{A} \cap E_J^{-1}(z)| \leq \frac{2^{|J|}}{|\text{im}(E_J)|} \leq \frac{8 \cdot 2^{|J|}}{|G|},$$

$$|\mathcal{B} \cap E_{J^c}^{-1}(z)| \leq \frac{2^{|J^c|}}{|\text{im}(E_{J^c})|} \leq \frac{8 \cdot 2^{|J^c|}}{|G|}.$$

There are at most $\varepsilon|G|$ vectors in I by (4.4). Substituting these bounds and $|J| + |J^c| = N$ in the count gives

$$|R \cap E^{-1}(\mathbf{0})| \leq |I| \frac{8 \cdot 2^{|J|}}{|G|} \cdot \frac{8 \cdot 2^{|J^c|}}{|G|} \leq 64\varepsilon \frac{2^N}{|G|}. \quad (4.5)$$

Compare with the whole zero fiber. Apply Lemma 30 to the full matrix E and output $\mathbf{0}$. There are at most $|G|$ outputs, so

$$|E^{-1}(\mathbf{0})| = \frac{2^N}{|\text{im}(E)|} \geq \frac{2^N}{|G|}. \quad (4.6)$$

Dividing the bound in (4.5) by this lower bound yields

$$\frac{|R \cap E^{-1}(\mathbf{0})|}{|E^{-1}(\mathbf{0})|} \leq \frac{64\varepsilon 2^N / |G|}{2^N / |G|} = 64\varepsilon.$$

□

The bound on the fraction covered by one rectangle now gives a lower bound on the number of rectangles, and hence on the DNNF size.

The DNNF lower bound for $\neg L_n$

✓ FinalCover: DNNF_lower_bound_size

Lemma 32. Every DNNF computing $\neg L_n$ has size $2^{\Omega(n^2)}$.

Proof. Suppose a DNNF of size s computes $\neg L_n$. By Lemma 28, the zero-input set of L_n is the union of balanced zero-rectangles $R_1, \dots, R_r$, with $r = O(s)$. Their intersections with $E^{-1}(\mathbf{0})$ cover $E^{-1}(\mathbf{0})$. The

rectangles may overlap, so we upper-bound the union by the sum of their sizes. Applying Lemma 31 to each rectangle gives

$$|E^{-1}(\mathbf{0})| = \left| \bigcup_{t=1}^r R_t \cap E^{-1}(\mathbf{0}) \right| \leq \sum_{t=1}^r |R_t \cap E^{-1}(\mathbf{0})| \leq 64\epsilon r |E^{-1}(\mathbf{0})|.$$

The zero fiber is nonempty by (4.6). Dividing by $|E^{-1}(\mathbf{0})|$ gives a lower bound on the number of rectangles:

$$1 \leq 64\epsilon r \implies r \geq \frac{1}{64\epsilon}. \quad (4.7)$$

Since $r = O(s)$, the circuit size is at least a constant times r . Using $\epsilon = 2^{-\Omega(n^2)}$ gives

$$s = \Omega(r) = \Omega(1/\epsilon) = 2^{\Omega(n^2)}.$$

□

Together with the small d-DNNF from Lemma 9, this proves Lemma 2.

5 Corollaries

Each corollary uses the function L_n from Lemma 2 or its complement. Throughout this section, setting $m = 2^{cn}$ for a sufficiently large fixed c turns the bounds $2^{O(n)}$ and $2^{\Omega(n^2)}$ into $O(m)$ and $m^{\Omega(\log m)}$.

5.1 Logic circuits

We use the standard notion of succinctness [24]: a language $\mathcal{A}$ is *at least as succinct as* $\mathcal{B}$ if every representation in $\mathcal{B}$ has an equivalent representation in $\mathcal{A}$ of polynomially bounded size. It is *strictly more succinct* if this holds in one direction but not the other.

Internal negation. Allowing internal NOT gates in a d-DNNF gives the language of *deterministic decomposable circuits* (d-D) [54]. Wachter and Haenni first proposed this language as *cd-PDAG* in 2006 and asked whether it is strictly more succinct than d-DNNF [78]. The same extension later appeared as *d-DNNF⁺* [39] and *partitioned-operation graphs* (POGs) [11]. These languages translate into one another with polynomial overhead.

d-D versus DNNF

✓ InternalNegation: internal_negation_separation

Corollary 33. *For arbitrarily large m , there are d-D circuits of size $O(m)$ whose functions require DNNFs of size $m^{\Omega(\log m)}$. Consequently, d-DNNF is strictly less succinct than d-D.*

Proof. Add a root NOT gate to the small d-DNNF of Lemma 2; the same lemma gives the DNNF lower bound. The reverse simulation holds because every d-DNNF is a d-D circuit. □

Boolean decomposable And-Sum circuits [31, 57] also simulate d-D, expressing negation as $1 - f$, and hence admit the same separation from arbitrary DNNFs. This includes structured decomposable And-Sum circuits (st-DASCs): choosing one common variable order for the term OBDDs in Lemma 8 makes the small circuit respect a single variable tree.

Unambiguous OBDDs. An *unambiguous OBDD* reads variables in a fixed order and allows nondeterministic choices, with at most one accepting computation per assignment. We measure OBDD size by its number of nodes.

Bollig and Buttkus asked in 2018 whether polynomial-size unambiguous OBDDs are closed under negation [7]. Nonclosure already follows from Vinall-Smeeth's 2024 separation [76]: his unambiguous DNF witness gives a small unambiguous OBDD, while its complement is hard for every structured DNNF. Here the lower bound holds for every DNNF.

Corollary 34. *For arbitrarily large m , there are Boolean functions f with unambiguous OBDDs of size $O(m)$ in every variable order. Every DNNF for $\neg f$ has size $m^{\Omega(\log m)}$. In particular, unambiguous OBDDs are not polynomially closed under negation.*

Proof. Fix any variable order. Guess one substituted outer term of L_n and run its small OBDD from Lemma 8. Distinct terms cannot both accept by Lemma 4, so this gives an unambiguous OBDD of size $2^{O(n)}$, as counted in Lemma 9. Lemma 2 gives the complement's DNNF lower bound $2^{\Omega(n^2)}$.

The decision-node conversion in Lemma 9, with choice nodes replaced by OR gates, converts an unambiguous OBDD to a DNNF with polynomial overhead. A polynomial-size negation operation would therefore contradict the lower bound. $\square$

5.2 Arithmetic circuits

A *monotone arithmetic circuit* uses variables, nonnegative constants, and binary addition and multiplication gates. A circuit is *syntactically multilinear* if the inputs to each multiplication gate use disjoint sets of variables. This restricts the computation, not just the polynomial at its output.

Subtraction can yield exponential savings over monotone arithmetic circuits [17, 75]. We use subtraction to complement a monomial set, with 0/1 coefficients on both sides. The product $\prod_{i=1}^N (1 + x_i)$ contains every multilinear monomial in $x_1, \dots, x_N$ exactly once, including the constant monomial 1. Subtracting a multilinear polynomial with 0/1 coefficients therefore leaves exactly its missing monomials. We ask whether this operation preserves polynomial monotone circuit size.

Missing monomials

✓ MissingMonomials: missing_monomials

Corollary 35. *For arbitrarily large m , there is a multilinear polynomial G in $N = O((\log m)^2)$ variables with 0/1 coefficients and monotone syntactically multilinear circuits of size $O(m)$. Every monotone arithmetic circuit whose monomials are precisely the multilinear monomials absent from G has size $m^{\Omega(\log m)}$, regardless of their positive coefficients.*

In particular, for this G , the polynomial

$$H(x) = \prod_{i=1}^N (1 + x_i) - G(x)$$

has 0/1 coefficients and a syntactically multilinear circuit of size $O(m)$ with one subtraction at the root, whereas every monotone arithmetic circuit for H has size $m^{\Omega(\log m)}$.

Proof. Put $N = 60n^2$ and encode each assignment $a \in \{0, 1\}^N$ by $m_a = \prod_{i:a_i=1} X_i \prod_{i:a_i=0} \bar{X}_i$, where $X_i, \bar{X}_i$ are independent variables. At $(X, \bar{X}) = (a, 1 - a)$, only m_a evaluates to one; all other assignment monomials evaluate to zero. This encoding will let us recover a DNNF. Write

$$P_n = \sum_{a:L_n(a)=1} m_a, \quad Q_n = \sum_{a:L_n(a)=0} m_a, \quad P_n + Q_n = T_n = \prod_{i=1}^N (X_i + \bar{X}_i).$$

The unambiguous OBDD from Corollary 34 gives a monotone syntactically multilinear circuit for P_n of size $2^{O(n)}$: complete each accepting path to read all positions, replace decisions by weighted sums using X_i and $\bar{X}_i$, and replace nondeterministic choices by sums. Unambiguity gives coefficient one on every accepted assignment monomial. For every i , rename X_i to x_i and set $\bar{X}_i$ to 1, defining

$$G_n(x) = P_n[X_i \leftarrow x_i, \bar{X}_i \leftarrow 1], \quad H_n(x) = Q_n[X_i \leftarrow x_i, \bar{X}_i \leftarrow 1].$$

Then G_n has the required small circuit, and H_n consists of the monomials missing from G_n .

For the lower bound, consider any monotone circuit whose output has exactly the monomials of Q_n , with positive coefficients. After pruning unused and zero subcircuits, every product combines disjoint sets of variable pairs: otherwise an output monomial would contain two variables from one pair, counting repetitions, with no cancellation possible. Replacing sums by OR, products by AND, positive constants by true, and the paired variables by opposite literals therefore gives a DNNF for $\neg L_n$ without increasing size. By Lemma 2, this proves the paired-variable lower bound $2^{\Omega(n^2)}$, regardless of the positive coefficients.

Finally, a monotone circuit for a multilinear polynomial is syntactically multilinear after pruning, since a repeated variable at a product could not cancel. The conversion of Broadrick, Zhang, and Van den Broeck [12, Lemma 3] turns such a circuit of size s into one of size $O(Ns)$ for its assignment encoding. It only inserts products with missing barred variables, preserving coefficients and monotonicity. Applied to any positive reweighting of H_n , this produces a positive reweighting of Q_n . Since $N = 60n^2$, the paired-variable lower bound gives $Ns = 2^{\Omega(n^2)}$ and hence $s = 2^{\Omega(n^2)}$. $\square$

5.3 Probabilistic circuits

A probabilistic circuit (PC) represents a probability distribution by sums and products of univariate functions [20]. Here the variables are Boolean. A circuit is *nonnegative* if its weights and input functions are nonnegative, and *decomposable* if the factors of each product use disjoint sets of variables. Loconte et al. [49, Theorem 1] prove an exponential separation between structured-decomposable PCs that allow subtraction and those restricted to nonnegative weights and inputs. We show that one subtraction, even from a constant, can give superpolynomial savings for general decomposable circuits.¹³

Probabilistic circuit subtraction

✓ ProbabilisticSubtraction: probabilistic_subtraction

Corollary 36. *For arbitrarily large m , there is a nonnegative structured-decomposable circuit C of size $O(m)$ such that the probability distribution $p(x) \propto 1 - C(x)$ is uniform on its support. Every nonnegative decomposable probabilistic circuit representing this distribution has size $m^{\Omega(\log m)}$.*

Proof. Construct the small arithmetic circuit for P_n using the same variable order for all term OBDDs. Its products therefore respect one common variable tree. Replace X_i and $\bar{X}_i$ by the indicator functions $1[x_i = 1]$ and $1[x_i = 0]$, respectively. This gives a nonnegative structured-decomposable circuit computing L_n , of the same size $2^{O(n)}$. One subtraction gives

$$p_n(x) \propto 1 - L_n(x).$$

Since $L_n(0) = F_n(0) = 0$, the complement has nonempty support. The distribution is therefore well defined and uniform on the assignments rejected by L_n .

In any nonnegative decomposable probabilistic circuit for p_n , replace each univariate input function by its Boolean support, discard zero-weight terms, and replace sums and products by OR and AND. The result is a DNNF for the support of p_n , namely $\neg L_n$, with only constant-factor overhead. Lemma 2 therefore gives the lower bound $2^{\Omega(n^2)}$. $\square$

Squared structured-decomposable PCs and their generalizations [49, 50, 79, 81] can therefore be superpolynomially more succinct than unstructured nonnegative decomposable PCs. Indeed, the circuit C constructed above computes a Boolean indicator, so $(1 - C(x))^2 = 1 - C(x)$. The standard structured squaring construction has only polynomial overhead, so the quasipolynomial separation survives [49].

5.4 Automata and grammars

An unambiguous finite automaton may make nondeterministic choices, but has at most one accepting computation on each word. Complementing an unambiguous finite automaton can require a superpolynomial increase in size, even when the output may be nondeterministic [32, 62].

¹³The general decomposable separation may also follow from the construction of Cavalar et al. [17, Theorem 1.1]. However, this implication has not previously been stated in the literature.

We allow the complement a more expressive representation: a context-free grammar. Even with this freedom, the complement of a language with a small unambiguous finite automaton can require a large description.

A right-linear grammar has rules $A \rightarrow bB$, $A \rightarrow b$, or $A \rightarrow \epsilon$, where A, B are nonterminals and b is an input symbol. It is unambiguous if each generated word has one parse tree. We count grammar size as the number of rules plus the number of symbol occurrences on their right-hand sides.

For infinite languages, an unambiguous context-free language can have a complement that is not context-free [35]. At a fixed finite length, a grammar for the complement always exists; the question is its size. Menger and Vinall-Smeeth left open whether complementation is hard for unambiguous context-free grammars over finite languages [52, Section 5]. We obtain a quasipolynomial lower bound even for right-linear grammars, allowing the complement grammar to be ambiguous.

Grammar complementation

✓ GrammarComplementation: grammar_complementation

Corollary 37. *For arbitrarily large m , there is a language $\mathcal{L} \subseteq \{0, 1\}^N$, with $N = O((\log m)^2)$, that has an unambiguous right-linear grammar of size $O(m)$. Every context-free grammar H that represents the complement of $\mathcal{L}$ on words of length N , that is,*

$$L(H) \cap \{0, 1\}^N = \{0, 1\}^N \setminus \mathcal{L},$$

has size $m^{\Omega(\log m)}$. The grammar H may be ambiguous and behave arbitrarily at other word lengths.

Proof outline. Put $\mathcal{L}_n = \{x_1 \cdots x_N : L_n(x) = 1\}$, where $N = 60n^2$. Complete the term OBDDs from Corollary 34 so that every accepting path reads all N positions. Reading bits as symbols and guessing the term gives an unambiguous finite automaton of size $2^{O(n)}$. Viewing states as nonterminals and transitions as productions gives a right-linear grammar of the same asymptotic size. Accepting runs and parse trees correspond one to one, preserving unambiguity.

On words of length N , a context-free grammar of size h gives a DNNF of size $(h + N)^{O(1)}$ [41, 61]. First split long right-hand sides into binary productions, with linear size overhead. A parsing chart then records which nonterminals can generate which intervals of the word. It joins adjacent intervals by AND gates and combines alternative derivations by OR gates [83]. The intervals joined by an AND gate have disjoint positions, giving decomposability. Unrolling the chart for as many rounds as it has entries accommodates empty productions and cycles with polynomial overhead. Applied to H , this circuit computes $\neg L_n$. Since $N = 60n^2$, Lemma 2 gives $h = 2^{\Omega(n^2)}$. $\square$

In database theory, factorized representations describe sets of tuples using shared unions and Cartesian products [56]. Kimelfeld, Martens, and Niewerth introduce an unnamed variant, identify it with grammars for fixed-length languages, and discuss complementation as an operation on these representations [45]. Through their size-preserving correspondence, Corollary 37 gives a quasipolynomial lower bound for complementing deterministic unnamed factorized relations, even when the output need not be deterministic.

Removing unambiguous choice. A deterministic pushdown automaton (DPDA, with one-way input) is a finite-state machine with a stack: it can push and pop symbols at the top of the stack, but cannot make nondeterministic choices [38]. The stack lets it recognize languages such as balanced parentheses. Every language recognized by a finite automaton also has a DPDA; the question here is how succinctly it can be represented. Can stack memory efficiently replace unambiguous choices? Schmidt listed this succinctness comparison as open in 1978 [72, Table 3.1]. The following corollary shows that even unrestricted stack memory cannot always avoid a superpolynomial increase in description size.

A DPDA may update the stack without consuming an input symbol (epsilon moves). Acceptance is by final state. Description size counts states, stack symbols, transitions and every symbol in a pushed string.

Corollary 38. *For arbitrarily large m , there is a finite binary language with an unambiguous finite automaton of size $O(m)$, whereas every DPDA recognizing that language has description size $m^{\Omega(\log m)}$.*

Proof outline. Ginsburg and Greibach [82] proved that deterministic context-free languages are closed under complementation. We use a quantitative version: a DPDA of description size d has a context-free grammar of size $d^{O(1)}$ for its complement. Our formalization constructs this grammar directly, using nonterminals that summarize parts of a run.

For example, a popping nonterminal describes the input words that take the machine from a state p with a stack segment γ to a state q just after removing that segment. The stack below it is untouched. Other nonterminals describe rejecting runs. Grammar rules combine these summaries: when the top symbol is popped, the run splits into the part that removes it and the part that processes the remaining segment. Rules for a single stack symbol follow the machine’s transitions. If the machine cannot read the next letter, including because it loops on epsilon moves, the rejecting nonterminal generates that letter followed by any word. Empty-input cases are recorded separately.

Only single stack symbols, suffixes of pushed strings, and the empty segment are needed, giving $O(d)$ segments. Combining these with states and the finitely many summary types gives polynomially many nonterminals and rules. Lean verifies that the summaries generate precisely the words they describe, that the start symbol generates the complement, and that the grammar has polynomial size.

For a DPDA recognizing $\mathcal{L}_n$, the preceding grammar lower bound therefore forces $d^{O(1)} \geq 2^{\Omega(n^2)}$, and hence $d = 2^{\Omega(n^2)}$. The preceding proof also supplies an unambiguous finite automaton of size $2^{O(n)}$. $\square$

6 Conclusion

Negation can turn a compact d-DNNF into a function requiring quasipolynomially larger DNNFs. Dropping determinism from the output does not avoid this increase. The proof combines an unambiguous DNF, a four-bit function, and a linear transformation: together they preserve a small representation of the function while obstructing every balanced rectangle cover of its zeros. Because the obstruction concerns the set of zeros, it also applies to representations that assign arbitrary positive weights to the complement. This connects the logical separation to the arithmetic and probabilistic consequences.

The separation also has consequences for automata and grammars. A small unambiguous finite automaton can recognize a language whose complement requires a quasipolynomially larger context-free grammar, even allowing ambiguity. Recognizing the original language deterministically can also require a quasipolynomially larger description, even with an unbounded stack. Through the grammar correspondence, the complement lower bound also applies to deterministic unnamed factorized relations.

Readers may wonder how our result relates to decision-DNNFs, which restrict disjunctions to variable decisions [37]. Appendix C shows, using existing results, that their negations have polynomial-size general d-DNNFs, so our lower bound does not apply to these inputs. It also shows that matching quasipolynomial upper and lower bounds for negation within decision-DNNF follow from existing results in the literature.

The precise cost of negating a d-DNNF remains open for both DNNF and d-DNNF outputs. Our quasipolynomial lower bound applies to both. For each output language, neither a matching quasipolynomial upper bound nor an exponential lower bound is known. The answers may differ: negations could potentially admit quasipolynomial-size DNNFs while sometimes requiring exponentially large d-DNNFs.

Acknowledgments

We thank the organizers and participants of the 2026 Dagstuhl seminar *Knowledge Compilation in Artificial Intelligence, Databases, and Formal Methods* for valuable discussions on the problem of negating d-DNNF. We also thank Todd Millstein for feedback on the Lean formalization and Steven Holtzen for helping provide core circuit definitions in Lean.

References

- [1] A. Amarilli. *Query Evaluation: Enumeration, Maintenance, Reliability*. Habilitation à diriger des recherches, Institut Polytechnique de Paris, 2023. [HAL tel-04065298](#)
- [2] A. Amarilli. *A List of Open Questions*, entry “PTIME complementation of d-DNNF”. [a3nm.net/work/research/questions](#), accessed 2026-09-13.
- [3] A. Amarilli, F. Capelli. *Tractable Circuits in Database Theory*. SIGMOD Record 53(2):6–20, 2024. doi:10.1145/3685980.3685982
- [4] A. Amarilli, M. Monet, P. Raphaël, S. Salvati. *On the Complexity of Language Membership for Probabilistic Words*. STACS 2026, LIPIcs 364, 5:1–5:21. doi:10.4230/LIPIcs.STACS.2026.5
- [5] K. Balodis, S. Ben-David, M. Göös, S. Jain, R. Kothari. *Unambiguous DNFs and Alon–Saks–Seymour*. FOCS 2021, 116–124. doi:10.1109/FOCS52979.2021.00020
- [6] P. Beame, J. Li, S. Roy, D. Suciú. *Lower Bounds for Exact Model Counting and Applications in Probabilistic Databases*. UAI 2013. [Author’s version](#).
- [7] B. Bollig, M. Buttkus. *On the Relative Succinctness of Sentential Decision Diagrams*. Theory of Computing Systems 63(6):1250–1277, 2019. doi:10.1007/s00224-018-9904-z. Preprint, February 2018: [arXiv:1802.04544v1](#).
- [8] B. Bollig, M. Fahrenholtz. *On the Relation Between Structured d-DNNFs and SDDs*. Theory of Computing Systems 65(2):274–295, 2021. doi:10.1007/s00224-020-10003-y
- [9] B. Bollig, I. Wegener. *Asymptotically Optimal Bounds for OBDDs and the Solution of Some Basic OBDD Problems*. ECCC Report TR99-048, 1999. [pdf](#).
- [10] S. Bova, F. Capelli, S. Mengel, F. Slivovsky. *Knowledge Compilation Meets Communication Complexity*. IJCAI 2016, 1008–1014. [pdf](#)
- [11] R. E. Bryant, W. Nawrocki, J. Avigad, M. J. H. Heule. *Certified Knowledge Compilation with Application to Verified Model Counting*. SAT 2023, 6:1–6:20. doi:10.4230/LIPIcs.SAT.2023.6
- [12] O. Broadrick, H. Zhang, G. Van den Broeck. *Polynomial Semantics of Tractable Probabilistic Circuits*. UAI 2024, PMLR 244:418–429. [Proceedings](#), [arXiv version](#).
- [13] F. Capelli. *Algorithmic Applications of Knowledge Compilation*. Habilitation à diriger des recherches, Université d’Artois, defended 15 June 2026. [florent.capelli.me/publi/hdr.pdf](#)
- [14] F. Capelli, Y. Choi, S. Mengel, M. Muñoz, and G. Van den Broeck. *A Canonical Generalization of OBDD*. SAT 2026, LIPIcs 377, 10:1–10:19. doi:10.4230/LIPIcs.SAT.2026.10
- [15] O. Čeppek, J. Glišić. *Boolean Nearest Neighbor Language in the Knowledge Compilation Map*. KR 2025, 240–249. doi:10.24963/kr.2025/24
- [16] C. Carlet. *Boolean Functions for Cryptography and Coding Theory*. Cambridge University Press, 2021. [Author’s version](#).
- [17] B. Cavalar, T. B. Fabris, P. Mukhopadhyay, S. Srinivasan, A. Yehudayoff. *Negations Are Powerful Even in Small Depth*. STOC 2026, 2071–2082. doi:10.1145/3798129.3800911, [arXiv version](#).
- [18] A. Chattopadhyay, M. Koucký, B. Loff, S. Mukhopadhyay. *Simulation Theorems via Pseudo-random Properties*. Computational Complexity 28:617–659, 2019. doi:10.1007/s00037-019-00190-7
- [19] A. Chattopadhyay, R. Datta, P. Mukhopadhyay. *Lower Bounds for Monotone Arithmetic Circuits via Communication Complexity*. STOC 2021, 786–799. doi:10.1145/3406325.3451069

- [20] Y. Choi, A. Vergari, G. Van den Broeck. *Probabilistic Circuits: A Unifying Framework for Tractable Probabilistic Models*. Technical report, 2020. [pdf](#)
- [21] A. Darwiche. *New Advances in Compiling CNF to Decomposable Negation Normal Form*. ECAI 2004. [Proceedings](#).
- [22] A. Darwiche. *SDD: A New Canonical Representation of Propositional Knowledge Bases*. IJCAI 2011, 819–826. [doi:10.5591/978-1-57735-516-8/IJCAI11-143](#)
- [23] A. Darwiche, P. Marquis. *A Perspective on Knowledge Compilation*. IJCAI 2001, 175–182. [ijcai.org](#)
- [24] A. Darwiche, P. Marquis. *A Knowledge Compilation Map*. JAIR 17, 2002. [arXiv:1106.1819](#)
- [25] M. Chavira, A. Darwiche. *On Probabilistic Inference by Weighted Model Counting*. Artificial Intelligence 172(6–7):772–799, 2008. [doi:10.1016/j.artint.2007.11.002](#)
- [26] M. Chavira, A. Darwiche, M. Jaeger. *Compiling Relational Bayesian Networks for Exact Inference*. International Journal of Approximate Reasoning 42(1–2):4–20, 2006. [doi:10.1016/j.ijar.2005.10.001](#)
- [27] A. de Colnet. *Hard Functions in Knowledge Compilation: from Lower Bounds to Applications*. PhD thesis, Université d’Artois, 2022. [theses.fr/2022ARTO0409](#)
- [28] D. Deutch, N. Frost, B. Kimelfeld, M. Monet. *Computing the Shapley Value of Facts in Query Answering*. SIGMOD 2022, 1570–1583. [doi:10.1145/3514221.3517912](#)
- [29] H. Fargier, P. Marquis. *Disjunctive Closures for Knowledge Compilation*. Artificial Intelligence 216:129–162, 2014. [doi:10.1016/j.artint.2014.07.004](#)
- [30] D. Fierens, G. Van den Broeck, J. Renkens, D. Shterionov, B. Gutmann, I. Thon, G. Janssens, L. De Raedt. *Inference and Learning in Probabilistic Logic Programs Using Weighted Boolean Formulas*. Theory and Practice of Logic Programming 15(3):358–401, 2015. [doi:10.1017/S1471068414000076](#)
- [31] J. R. L. Gonçalves, D. Mauá. *On the Relationship between And-Sum Circuits and Deterministic Boolean Circuits*. Workshop on Tractable Probabilistic Modeling (TPM), 2025. [OpenReview](#), [pdf](#).
- [32] M. Göös, S. Kiefer, W. Yuan. *Lower Bounds for Unambiguous Automata via Communication Complexity*. ICALP 2022, LIPIcs 229, 126:1–126:13. [doi:10.4230/LIPIcs.ICALP.2022.126](#)
- [33] V. Guruswami. *Bridging Shannon and Hamming: List Error-Correction with Optimal Rate*. Proceedings of the International Congress of Mathematicians, 2010. [Author’s version](#).
- [34] W. H. Haemers. *Hoffman’s Ratio Bound*. Linear Algebra and its Applications 617:215–219, 2021. [doi:10.1016/j.laa.2021.02.010](#)
- [35] T. N. Hibbard, J. Ullian. *The Independence of Inherent Ambiguity From Complementedness Among Context-Free Languages*. Journal of the ACM 13(4):588–593, 1966. [doi:10.1145/321356.321366](#)
- [36] S. Holtzen, G. Van den Broeck, T. Millstein. *Scaling Exact Inference for Discrete Probabilistic Programs*. Proceedings of the ACM on Programming Languages 4(OOPSLA), Article 140, 2020. [doi:10.1145/3428208](#)
- [37] J. Huang, A. Darwiche. *The Language of Search*. Journal of Artificial Intelligence Research 29:191–219, 2007. [doi:10.1613/jair.2097](#)
- [38] J. E. Hopcroft, R. Motwani, J. D. Ullman. *Introduction to Automata Theory, Languages, and Computation*. Third edition, Pearson, 2007. [Publisher](#).
- [39] A. Jha, D. Suciu. *Knowledge Compilation Meets Database Theory: Compiling Queries to Decision Diagrams*. ICDT 2011, 162–173. [doi:10.1145/1938551.1938574](#)

- [40] A. Jha, D. Suciu. *Knowledge Compilation Meets Database Theory: Compiling Queries to Decision Diagrams*. Theory of Computing Systems 52(3):403–440, 2013. doi:10.1007/s00224-012-9392-5
- [41] J. C. Jung, P. Barahona, G. Katsirelos, T. Walsh. *Two Encodings of DNNF Theories*. ECAI Workshop on Inference Methods Based on Graphical Structures of Knowledge, 2008. pdf
- [42] R. Kiesel, T. Eiter. *Knowledge Compilation and More with SharpSAT-TD*. KR 2023, 406–416. doi:10.24963/kr.2023/40
- [43] S. Khanna, S. Roy, V. Tannen. *Queries with Difference on Probabilistic Databases*. Proceedings of the VLDB Endowment 4(11):1051–1062, 2011. doi:10.14778/3402707.3402741
- [44] D. Kahdian, O. Broadrick, R. L. Geh, G. Van den Broeck. *Tractable Circuit Zoo*. circuitzoo.net, decision-DNNF negation entry, accessed 2026-09-14.
- [45] B. Kimelfeld, W. Martens, M. Niewerth. *A Formal Language Perspective on Factorized Representations*. ICDT 2025, LIPIcs 328, 20:1–20:20. doi:10.4230/LIPIcs.ICDT.2025.20
- [46] F. Koriche, J.-M. Lagniez, P. Marquis, S. Thomas. *Knowledge Compilation for Model Counting: Affine Decision Trees*. IJCAI 2013, 947–953. Proceedings
- [47] J.-M. Lagniez, P. Marquis. *An Improved Decision-DNNF Compiler*. IJCAI 2017, 667–673. doi:10.24963/ijcai.2017/93
- [48] Y. Lai, D. Liu, M. Yin. *New Canonical Representations by Augmenting OBDDs with Conjunctive Decomposition*. Journal of Artificial Intelligence Research 58:453–521, 2017. doi:10.1613/jair.5271
- [49] L. Loconte, A. M. Sladek, S. Mengel, M. Trapp, A. Solin, N. Gillis, A. Vergari. *Subtractive Mixture Models via Squaring: Representation and Learning*. ICLR 2024. arXiv:2310.00724
- [50] L. Loconte, S. Mengel, A. Vergari. *Sum of Squares Circuits*. AAAI 2025. arXiv:2408.11778
- [51] C. Muise, S. A. McIlraith, J. C. Beck, E. I. Hsu. *Dsharp: Fast d-DNNF Compilation with sharpSAT*. Canadian AI 2012, 356–361. doi:10.1007/978-3-642-30353-1_36
- [52] S. Mengel, H. Vinall-Smeeth. *A Lower Bound on Unambiguous Context Free Grammars via Communication Complexity*. PODS 2025. doi:10.1145/3725225
- [53] M. Monet. *Solving a Special Case of the Intensional vs Extensional Conjecture in Probabilistic Databases*. PODS 2020, 149–163. doi:10.1145/3375395.3387642
- [54] M. Monet, D. Olteanu. *Towards Deterministic Decomposable Circuits for Safe Queries*. AMW 2018, CEUR Workshop Proceedings 2100. pdf
- [55] R. O’Donnell. *Analysis of Boolean Functions*. Cambridge University Press, 2014. Revised author’s edition, 2021.
- [56] D. Olteanu, J. Závodný. *Size Bounds for Factorised Representations of Query Results*. ACM Transactions on Database Systems 40(1):2:1–2:44, 2015. doi:10.1145/2656335
- [57] R. Onaka, K. Nakamura, M. Nishino, N. Yasuda. *An And-Sum Circuit with Signed Edges That Is More Succinct than SDD*. AAAI 39(14):15100–15108, 2025. doi:10.1609/aaai.v39i14.33656
- [58] C. Pabbaraju. *Optimal Unambiguous DNFs and Alon–Saks–Seymour*. August 2026. arXiv:2608.02533
- [59] U. Oztok, A. Darwiche. *A Top-Down Compiler for Sentential Decision Diagrams*. IJCAI 2015, 3141–3148. Proceedings.
- [60] K. Pipatsrisawat, A. Darwiche. *New Compilation Languages Based on Structured Decomposability*. AAAI 2008, 517–522. Proceedings.

- [61] C.-G. Quimper, T. Walsh. *Decomposing Global Grammar Constraints*. CP 2007, LNCS 4741, 590–604. doi:10.1007/978-3-540-74970-7_42
- [62] M. Raskin. *A Superpolynomial Lower Bound for the Size of Non-Deterministic Complement of an Unambiguous Automaton*. ICALP 2018, LIPIcs 107, 138:1–138:11. doi:10.4230/LIPIcs.ICALP.2018.138
- [63] I. Razgon. *On the Read-Once Property of Branching Programs and CNFs of Bounded Treewidth*. Algorithmica 75(2):277–294, 2016. doi:10.1007/s00453-015-0059-x
- [64] D. Sieling, I. Wegener. *NC-Algorithms for Operations on Binary Decision Diagrams*. Parallel Processing Letters 3(1):3–12, 1993. doi:10.1142/S0129626493000022.
- [65] R. P. Stanley. *Enumerative Combinatorics, Volume 1*. Second edition, Cambridge University Press, 2012. [Author’s version](#).
- [66] T. Sang, P. Beame, H. Kautz. *Performing Bayesian Inference by Weighted Model Counting*. AAAI 2005, 475–481. [Proceedings](#).
- [67] B. Selman, H. Kautz. *Knowledge Compilation Using Horn Approximations*. AAAI 1991, 904–909. [pdf](#)
- [68] A. Shih, G. Van den Broeck, P. Beame, A. Amarilli. *Smoothing Structured Decomposable Circuits*. Advances in Neural Information Processing Systems 32 (NeurIPS), 2019. [Proceedings](#).
- [69] A. Shpilka, A. Yehudayoff. *Arithmetic Circuits: A Survey of Recent Results and Open Questions*. Foundations and Trends in Theoretical Computer Science 5(3–4):207–388, 2010. doi:10.1561/04000000039
- [70] S. Subbarayan, L. Bordeaux, Y. Hamadi. *Knowledge Compilation Properties of Tree-of-BDDs*. AAAI 2007, 502–507. [Proceedings](#)
- [71] D. Suciu. *Probabilistic Databases for All*. PODS 2020, 19–31. doi:10.1145/3375395.3389129
- [72] E. M. Schmidt. *Succinctness of Descriptions of Context-Free, Regular, and Finite Languages*. Ph.D. thesis, Cornell University; circulated as DAIMI Report PB-84, Aarhus University, 1978. doi:10.7146/dpb.v7i84.6500
- [73] D. Suciu, D. Olteanu, C. Ré, C. Koch. *Probabilistic Databases*. Synthesis Lectures on Data Management, 2011. doi:10.1007/978-3-031-01879-4
- [74] T. Tao. *Mathematics in the Age of AI*. August 2026. [arXiv:2608.16753](#)
- [75] L. G. Valiant. *Negation Can Be Exponentially Powerful*. Theoretical Computer Science 12:303–314, 1980; preliminary version STOC 1979, 189–196. doi:10.1016/0304-3975(80)90060-2
- [76] H. Vinall-Smeeth. *Structured d -DNNF Is Not Closed Under Negation*. IJCAI 2024. [arXiv:2402.04832](#)
- [77] J. Vlasselaer, J. Renkens, G. Van den Broeck, L. De Raedt. *Compiling Probabilistic Logic Programs into Sentential Decision Diagrams*. Workshop on Probabilistic Logic Programming (PLP), 2014. [pdf](#)
- [78] M. Wachter, R. Haenni. *Propositional DAGs: A New Graph-Based Language for Representing Boolean Functions*. KR 2006, 277–285. [pdf](#)
- [79] B. Wang, G. Van den Broeck. *On the Relationship Between Monotone and Squared Probabilistic Circuits*. AAAI 2025. [arXiv:2408.00876](#)
- [80] I. Wegener. *Branching Programs and Binary Decision Diagrams: Theory and Applications*. SIAM Monographs on Discrete Mathematics and Applications, 2000. doi:10.1137/1.9780898719789
- [81] P. Zuidberg Dos Martires. *A Quantum Information Theoretic Approach to Tractable Probabilistic Models*. UAI 2025, PMLR 286:3013–3030. [Proceedings](#).

- [82] S. Ginsburg, S. A. Greibach. *Deterministic Context Free Languages*. Information and Control 9(6):620–648, 1966; preliminary version SWCT (FOCS) 1965, 203–220. doi:10.1016/S0019-9958(66)80019-0
- [83] S. M. Shieber, Y. Schabes, and F. C. N. Pereira. *Principles and Implementation of Deductive Parsing*. Journal of Logic Programming 24(1-2):3-36, 1995. doi:10.1016/0743-1066(95)00035-I

A The Lean trust boundary

This appendix gives a self-contained formal statement of the nonclosure claim. Readers need to check that these definitions and the theorem statement express the intended mathematical claim. If they do, Lean’s verified proof establishes the claim, assuming the correctness of Lean’s kernel and its standard axioms `propext`, `Classical.choice`, and `Quot.sound`. The source code is available in the [Lean repository](#).

A.1 Nodes and circuits

A circuit with m nodes will number them $0, \dots, m - 1$. Lean writes `Fin m` for precisely this set of m positions. A value of type `Node Var m` describes the node stored at one position, where `Var` is the type of variable names:

Node types

Lean: TrustBoundary

```
inductive Node (Var : Type) (m : ℕ)
| const (b : Bool)
| lit (x : Var) (b : Bool)
| and (children : Finset (Fin m))
| or (children : Finset (Fin m))
```

The first constructor is a Boolean constant. The second is a literal: `lit x b` is true exactly when variable x has value b . Negation occurs nowhere else, so every circuit is in negation normal form. The last two constructors store the finite set of child positions of an AND or OR node; `Finset (Fin m)` is the type of finite sets of positions. Thus both kinds of internal node have arbitrary finite fan-in, and, since a set has no repeated element, no position is a child of the same node twice.

Three functions describe one node in terms of its child positions. The first returns the set of those positions, empty for a constant or a literal:

Child positions

Lean: TrustBoundary

```
variable {Var : Type} {m : ℕ}

def children : Node Var m → Finset (Fin m)
| const _ => ∅
| lit _ _ => ∅
| and children => children
| or children => children

abbrev ChildPosition (n : Node Var m) := {j // j ∈ n.children}
```

The `variable` line introduces implicit parameters shared by the three definitions in this section. They live in the namespace `Node`. A value of type `ChildPosition n` packages a position together with proof that it belongs to `n.children`.

Truth. `Node.evalLocal` evaluates just one node. Its first input `v` assigns Boolean values to variables. Its second input `child` supplies the already computed truth value of each child.

Evaluating one node

Lean: TrustBoundary

```
def evalLocal (n : Node Var m) (v : Var → Bool)
  (child : ChildPosition n → Bool) : Bool :=
  match n with
  | const b => b
  | lit x b => v x == b
  | and _ => decide (∀ j, child j = true)
  | or _ => decide (∃ j, child j = true)
```

In the last two cases, j ranges over this node's child positions. The underscore in the pattern means that the set of children is not named again: n already fixes it. The operation `decide` returns the Boolean truth value of the finite quantifier: an AND node is true exactly when every child is, and an OR node exactly when some child is.

Variables. The local support function has the same shape. This time `child` supplies the set of variables mentioned at an actual child, and `Node.supportLocal` takes their union:

Variables of one node

Lean: TrustBoundary

```
def supportLocal [DecidableEq Var] (n : Node Var m)
  (child : ChildPosition n → Finset Var) : Finset Var :=
  match n with
  | const _ => ∅
  | lit x _ => {x}
  | and _
  | or _ => Finset.univ.biUnion child
```

`Finset Var` is the type of finite sets of variables. The assumption `[DecidableEq Var]`, that equality of variable names is decidable, is what Lean needs to form unions of such sets; it holds for $\mathbb{N}$. `Finset.univ` is the set of all packaged child positions of this node, and `biUnion child` takes the union of `child j` over its elements j . This is syntactic support, not semantic dependence: a circuit may mention a variable even if its output ultimately ignores that variable. DNNF decomposability is defined using this syntactic notion.

A finite shared circuit. A circuit is now a numbered table of nodes in topological order, together with one distinguished output position:

Circuit representation

Lean: TrustBoundary

```
structure Circuit (Var : Type) where
  nodeCount : ℕ
  node : Fin nodeCount → Node Var nodeCount
  children_lt : ∀ i, ∀ j ∈ (node i).children, j < i
  output : Fin nodeCount
```

If several parent nodes name the same position, they share one node rather than receiving separate copies. The strict decrease in `children_lt` prevents directed cycles. Consequently this is a shared directed acyclic graph, not an unfolded formula tree. The field `nodeCount` counts the stored positions, including unreachable ones. Circuit size counts edges plus one, so even a single constant or literal has size one:

```
def edgeCount (C : Circuit Var) : ℕ :=
  ∑ i, (C.node i).children.card
```

```
def size (C : Circuit Var) : ℕ := C.edgeCount + 1
```

The added one can be viewed as an output wire. Each child reference is counted once; sharing a child does not duplicate its internal edges. Removing unreachable nodes cannot increase size.

Truth at a position. The value of the node at position i is obtained by evaluating that node with the values of the positions below it, computed by the same function:

Recursive evaluation

Lean: TrustBoundary

```
variable {Var : Type}

def eval (C : Circuit Var) (v : Var → Bool) (i : Fin C.nodeCount) : Bool :=
  (C.node i).evalLocal v fun j ↦ C.eval v j.1
termination_by i.val
decreasing_by
  exact C.children_lt i j.1 j.2
```

Here $C.node\ i$ is the node at position i , and $fun\ j \mapsto C.eval\ v\ j.1$ supplies its local evaluation with recursively computed child values. The component $j.1$ is the child's position, and $j.2$ proves that it belongs to this node's children. The line `termination_by i.val` specifies the position's natural-number value as the decreasing quantity; `children_lt` uses the membership proof to establish $j.1 < i$ and justify the recursive call.

Variables at a position. The variables mentioned at or below a position are collected by the same recursion:

Recursive variable support

Lean: TrustBoundary

```
def support [DecidableEq Var] (C : Circuit Var) (i : Fin C.nodeCount) :
  Finset Var :=
  (C.node i).supportLocal fun j ↦ C.support j.1
termination_by i.val
decreasing_by
  exact C.children_lt i j.1 j.2
```

The same membership proof justifies each recursive support call. Consequently $C.support\ i$ is exactly the finite set of variables occurring at or below position i .

A.2 The computed function and the circuit classes

The computed function. The circuit computes a Boolean function by reading its distinguished output position:

The computed function

Lean: TrustBoundary

```
def Computes (C : Circuit Var) (f : (Var → Bool) → Bool) : Prop :=
  ∀ v, C.eval v C.output = f v
```

`Computes` says that the value at the output position equals $f\ v$ on every assignment.

Decomposability. At an AND node, every two distinct child subcircuits must mention disjoint sets of variables:

Decomposability

Lean: TrustBoundary

```
def IsDecomposable [DecidableEq Var] (C : Circuit Var) : Prop :=
  ∀ i children, C.node i = .and children →
    ∀ left ∈ children, ∀ right ∈ children, left ≠ right →
      Disjoint (C.support left) (C.support right)
```

Determinism. At an OR node, two different children must never both evaluate to true:

Determinism

Lean: TrustBoundary

```
def IsDeterministic (C : Circuit Var) : Prop :=
  ∀ i children, C.node i = .or children →
    ∀ left ∈ children, ∀ right ∈ children, left ≠ right →
      ∀ v, (C.eval v left && C.eval v right) = false
```

The expressions `.and children` and `.or children` abbreviate the corresponding `Node` constructors. Since `children` is a set, `left` and `right` with `left ≠ right` range over every pair of different children, and no child can be paired with a second copy of itself.

The two circuit classes

Lean: TrustBoundary

```
def IsDNNF [DecidableEq Var] (C : Circuit Var) : Prop :=
  C.IsDecomposable

def IsDeterministicDNNF [DecidableEq Var] (C : Circuit Var) : Prop :=
  C.IsDecomposable ∧ C.IsDeterministic
```

A.3 The exact Lean claim

The last declaration in the file states nonclosure using the definitions above:

The nonclosure statement

Lean: TrustBoundary

```
def NotClosedUnderNegation : Prop :=
  ∀ d M : ℕ,
    ∃ C : Circuit ℕ, C.IsDeterministicDNNF ∧
      ∀ D : Circuit ℕ, D.IsDNNF →
        D.Computes (fun v ↦ !C.eval v C.output) →
          M * C.size ^ d < D.size
```

Read the quantifiers from the outside inward. For every proposed exponent d and factor M , Lean supplies a d-DNNF C . Every DNNF D for its pointwise complement must then satisfy $|D| > M|C|^d$.

Why the quantifiers say non-closure

$$\begin{aligned} \text{polynomial closure: } \exists d, M \ \forall C \ \exists D : & \quad |D| \leq M|C|^d, \\ \text{its negation: } \forall d, M \ \exists C \ \forall D : & \quad |D| > M|C|^d. \end{aligned}$$

C ranges over d-DNNFs and D ranges over DNNFs computing their complements.

This monomial formulation captures every polynomial size bound. For any polynomial p , one can choose d and then enlarge M so that $p(s) \leq Ms^d$ for every integer $s \geq 1$, including the finitely many small sizes.

Two details complete the audit:

1. `fun v ↦ !C.eval v C.output` is the pointwise complement of the output of `C`: `!` is Boolean negation, so `D` must be false exactly where `C` is true, on every assignment $v : \mathbb{N} \rightarrow \text{Bool}$.
2. Both circuits use variable names in $\mathbb{N}$, so they receive assignments on the same domain. Each mentions only finitely many variables; `D` may mention others, but must still compute the complement on every assignment. Node positions are private to each circuit; the bound compares their sizes, counting edges plus one.

The exported theorem in `NegationNotClosed.lean` has exactly this type:

Proved nonclosure theorem	✓ Lean: <code>NegationNotClosed</code>
<code>theorem dDNNF_not_closed_under_negation : NotClosedUnderNegation</code>	

The proof is free to introduce other circuit representations and transport between them. Those constructions are checked by Lean’s kernel and add no assumptions to this theorem. For semantic auditing, the definitions and statement displayed above are the complete interface that matters.

Formalization statistics. The core formalization comprises 36 Lean modules and 10,269 source lines, including comments. With Mathlib precompiled, a fresh build took about 300 seconds on a single CPU thread to formally verify our non-closure claim.

B Proof details

B.1 Padding the hard function

We prove Theorem 1 assuming Lemma 2.

Proof. Choose constants $a, b > 0$ so that, for all sufficiently large n , Lemma 2 supplies a d-DNNF for L_n of size at most 2^{an} and a DNNF lower bound 2^{bn^2} for $\neg L_n$. Choose a fixed integer $c > a$. Set the padded input length to $m_n = 2^{cn}$ and put $q_n = m_n - N$. Add a block of q_n fresh bits $y \in \{0, 1\}^{q_n}$ that the function does not use:

$$H_n(x, y) = L_n(x).$$

For sufficiently large n , we have $m_n \geq N$, so $q_n \geq 0$. The same d-DNNF computes H_n , and

$$\text{d-DNNF-size}(H_n) \leq 2^{an} \leq m_n.$$

Now suppose C is a DNNF for $\neg H_n$. Fix every added variable y to 0, meaning the all-zero string 0^{q_n} . Substituting constants and simplifying gates preserves decomposability and cannot increase circuit size. The restricted circuit computes $\neg L_n$. Therefore

$$|C| \geq 2^{bn^2}.$$

Since $\log_2 m_n = cn$, this lower bound can be rewritten as

$$\begin{aligned} 2^{bn^2} &= 2^{b((\log_2 m_n)/c)^2} = 2^{(b/c^2)(\log_2 m_n)^2} = 2^{((b/c^2) \log_2 m_n) \log_2 m_n} \\ &= (2^{\log_2 m_n})^{(b/c^2) \log_2 m_n} = m_n^{(b/c^2) \log_2 m_n} = m_n^{\Omega(\log m_n)}. \end{aligned}$$

Thus the exponent of m_n is itself proportional to $\log m_n$, which is exactly the displayed quasipolynomial form.

To see the closure conclusion directly, suppose that a polynomial p bounded the size of every negated d-DNNF. For some fixed k , we have $p(s) \leq m_n^k$ whenever $s \leq m_n$ and n is sufficiently large. Applying this to the displayed d-DNNF for H_n would give a DNNF for $\neg H_n$ of size at most m_n^k . This contradicts the lower bound $m_n^{(b/c^2) \log_2 m_n}$ once $(b/c^2) \log_2 m_n > k$. $\square$

B.2 An algebraic derivation of the local inner products

We prove the local inner-product formulas in Lemma 15 without checking the sixteen parameters separately.¹⁴

Proof. Let $\delta_0(w, w')$ be 1 when $w = w' = 00$ and 0 otherwise. The two tables can be written as

$$\begin{aligned}\omega_1(w, w') &= \frac{1}{12}(1 - (-1)^{Q(w, w')}), \\ \omega_0(w, w') &= \frac{7}{12}\delta_0(w, w') + \frac{1}{48} + \frac{1}{48}(-1)^{Q(w, w')}.\end{aligned}\tag{B.1}$$

At $\xi = (00, 00)$, every dot product in the exponent is zero, so every entry of the parity vector is 1. The inner products are therefore the total weights from Definition 10:

$$\langle \omega_0, \text{par}_{(00,00)} \rangle = 1, \quad \langle \omega_1, \text{par}_{(00,00)} \rangle = 1.$$

Now suppose $\xi \neq (00, 00)$. In the formula for each inner product, the constant function contributes 0 by (3.4). The δ_0 term contributes only at $z = (00, 00)$, where the parity sign is 1. To calculate the contribution of $(-1)^Q$, write $\xi = (\xi', \xi'')$, where both entries are two-bit vectors. Adding ξ'' to w and ξ' to w' permutes the sixteen pairs. The dot products between these entries use two bits, while $\xi \cdot (w, w')$ uses all four. Expanding Q at the shifted pair gives

$$\begin{aligned}Q(w + \xi'', w' + \xi') &= w \cdot w' + \xi' \cdot w + \xi'' \cdot w' + \xi' \cdot \xi'' \\ &= Q(w, w') + \xi \cdot (w, w') + Q(\xi) \pmod{2}.\end{aligned}$$

Since addition is modulo two, moving $Q(\xi)$ to the other side gives the sign identity

$$(-1)^{Q(w, w') + \xi \cdot (w, w')} = (-1)^{Q(\xi)} (-1)^{Q(w + \xi'', w' + \xi')}.$$

The shifted pairs range over all of V , so their gadget outputs still contain the ten zeros and six ones shown in the table of Q . The sign $(-1)^{Q(\xi)}$ is independent of the pair, and therefore

$$\sum_{(w, w') \in V} (-1)^{Q(w, w') + \xi \cdot (w, w')} = (-1)^{Q(\xi)} (10 - 6) = 4(-1)^{Q(\xi)}.$$

The parity entries sum to 0, the parity entry at zero is 1, and the quadratic contribution is the sum just computed. Thus (B.1) gives

$$\begin{aligned}\langle \omega_1, \text{par}_\xi \rangle &= \frac{1}{12}(0 - 4(-1)^{Q(\xi)}) = -\frac{1}{3}(-1)^{Q(\xi)}, \\ \langle \omega_0, \text{par}_\xi \rangle &= \frac{7}{12} \cdot 1 + \frac{1}{48} \cdot 0 + \frac{1}{48} \cdot 4(-1)^{Q(\xi)} = \frac{7}{12} + \frac{1}{12}(-1)^{Q(\xi)}.\end{aligned}$$

Reading these two expressions according to whether $Q(\xi)$ is 0 or 1 gives the two remaining cases in the lemma. $\square$

B.3 From a DNNF to a balanced rectangle cover

Lemma 28 follows from Bova, Capelli, Mengel, and Slivovsky [10, Theorem 6]: a binary DNNF has a balanced rectangle cover of its satisfying assignments with at most as many rectangles as gates. We only need to accommodate arbitrary fan-in and unused input positions.

¹⁴Related calculations for quadratic Boolean functions can be found in Carlet [16, Section 6.1.15].

Preparing the circuit. Let D be a DNNF of size s computing $\neg L_n$. Interpret empty AND and OR gates as 1 and 0, propagate constants, bypass unary gates, and delete gates that do not reach the output. These operations preserve the function and decomposability without increasing size. If at most one variable remains, put it on one side of any balanced partition. The other positions are unrestricted, so at most one rectangle suffices. This also covers constant functions.

Replace each remaining gate with more than two inputs by a binary tree of gates of the same kind. At an AND gate this groups disjoint variable sets, so decomposability is preserved. Replacing a gate with k inputs takes $O(k)$ gates, and the sum of the original fan-ins is at most s . The resulting binary, constant-free DNNF has $O(s)$ nodes. The cited theorem gives a cover with $O(s)$ rectangles.

Keeping all N input positions. The theorem balances each partition over the variables mentioned by the circuit. Each side therefore has at most $2N/3$ positions already. Distribute the unused positions between the two sides so that each has between $N/3$ and $2N/3$ positions. This is possible: fill either side that has fewer than $N/3$ positions to that threshold, then distribute the rest without exceeding $2N/3$. Here $N = 60n^2$, so the thresholds are integers. Allow both values at every added position. This preserves the rectangle property and extends the cover to assignments on all $[N]$.

The resulting $O(s)$ balanced rectangles cover precisely the satisfying assignments of $\neg L_n$, which are the zero-inputs of L_n . $\square$

C Negation of decision-DNNFs

A decision-DNNF is a d-DNNF that restricts disjunctions to variable decisions $(x \wedge f) \vee (\neg x \wedge g)$, where neither branch contains the variable x [37, Definition 2]. An FBDD uses only these decisions and constants.

Beame, Li, Roy, and Suciú [6, Corollary 3.2] give a quasipolynomial-time conversion of decision-DNNFs into FBDDs. Swapping the terminal values then negates the FBDD without increasing its size. Thus a decision-DNNF of size s can be negated in time $s^{O(\log s)}$, with the output still a decision-DNNF.

The reader might wonder how our result affects decision-DNNF circuits. Our separation does not apply to these inputs: their negations have polynomial-size general d-DNNFs. Polynomial-time negation within decision-DNNF has been listed as open [27, 44, 48]. However, a matching quasipolynomial lower bound already follows by combining existing results.

Decision-DNNF negation

Proposition 39. *Let C be a decision-DNNF of size s computing a Boolean function f .*

- (i) *A general d-DNNF for $\neg f$ of size $O(s)$ can be constructed in $O(s)$ time.*
- (ii) *A decision-DNNF for $\neg f$ can be constructed in time $s^{O(\log s)}$. For arbitrarily large s , there are decision-DNNFs of size $O(s)$ whose negations require decision-DNNFs of size $s^{\Omega(\log s)}$.*

Proof. **General d-DNNF outputs.** Discard nodes that do not reach the output, then replace each conjunction by a binary tree of conjunctions. The construction adds only a constant number of edges per original edge, so it takes $O(s)$ time and produces a circuit of size $O(s)$. Construct both polarities of each gate, sharing them wherever they are reused. Complement constants and complement a decision by complementing its two branches. For a decomposable conjunction of g and h , use

$$\neg(g \wedge h) = \neg g \vee (g \wedge \neg h).$$

The disjunction is deterministic because its branches disagree on g , and the new conjunction is decomposable because g and h use disjoint variables. Each gate of the binary circuit takes constant work and contributes constantly many new edges, proving (i).

Decision-DNNF outputs. The upper bound is the FBDD simulation recalled above. For the matching lower bound, Razgon’s separation [63, Theorem 5], expressed in terms of the input circuit size, gives decision-DNNFs of arbitrarily large size s whose functions F are positive 2-CNFs and require FBDDs of size $s^{\Omega(\log s)}$. We show that a decision-DNNF for $\neg F$ would also give an FBDD for F with only polynomial overhead.

Suppose that $\neg F$ has a decision-DNNF of size t . Negating the clauses of F gives a DNF with two negative literals per term. Complementing every input variable turns it into a monotone 2-DNF without changing the decision-DNNF size. Beame, Li, Roy, and Suciú [6, Theorem 3.4] show that decision-DNNFs computing monotone 2-DNFs can be converted to FBDDs with polynomial overhead. Undoing the input complementation and swapping terminal values therefore gives an FBDD for F of size $t^{O(1)}$. Razgon’s lower bound therefore forces $t^{O(1)} \geq s^{\Omega(\log s)}$, and hence $t \geq s^{\Omega(\log s)}$, proving (ii). $\square$